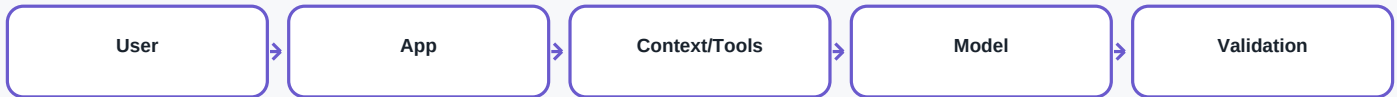


The AI application stack

WHAT TO LEARN - IN ORDER

- Learn to see an AI product as model + context + tools + deterministic code + storage + evals + operations, not "just a prompt".



LINEAR LEARNING PATH

1. Trace one request from user input to model output
2. Separate deterministic logic from model judgment
3. Name quality, latency, cost, and reliability as first-class constraints
4. Identify which components must be observable

DO THIS

Draw the architecture of one AI app you already know and label every non-model dependency.

MID-SENIOR CHECKPOINT

If the model is only 20% of the product, where should most production bugs be expected?

REQUIRED READING

Building LLM Applications for Production - Chip Huyen

Frames an LLM product as an end-to-end system with quality, latency, cost, and reliability constraints.

OFFICIAL / DEEP REFERENCE

Production Best Practices - OpenAI

Production checklist for reliability, latency, usage limits, and operational safeguards.

What happens when you type a URL?

WHAT TO LEARN - IN ORDER

- Build the end-to-end mental model for the web request path before studying any distributed component in isolation.



LINEAR LEARNING PATH

1. Resolve a name to an IP
2. Establish network/security transport
3. Send an HTTP request through intermediaries
4. Follow the request into application and data tiers

DO THIS

Use `curl -v` on a public site and identify DNS, connection, TLS, request, and response stages.

MID-SENIOR CHECKPOINT

At which stages can latency or failure occur before your application code even runs?

REQUIRED READING

Client-Server Overview - MDN

Explains the basic request-response relationship between clients, servers, applications, and data stores.

OFFICIAL / DEEP REFERENCE

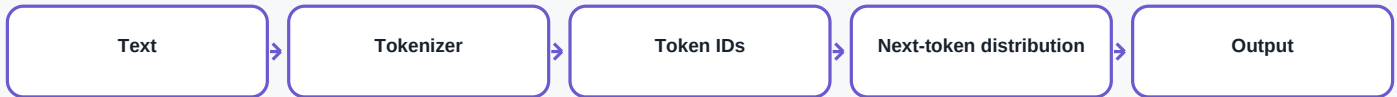
Overview of HTTP - MDN

Explains methods, headers, responses, intermediaries, caching, and stateless request semantics.

Tokens and autoregressive generation

WHAT TO LEARN - IN ORDER

- Understand what the model actually consumes and why output length dominates interactive generation latency.



LINEAR LEARNING PATH

1. Tokenize text and inspect token IDs
2. Understand next-token conditional probability
3. Compare greedy, temperature, and top-p decoding
4. Connect output tokens to cost and latency

DO THIS

Tokenize the same paragraph with two tokenizers and compare token counts.

MID-SENIOR CHECKPOINT

Why can two prompts with similar character length have very different cost?

REQUIRED READING

Deep Dive into Text Generation - Hugging Face

Explains autoregressive generation and the mechanics behind next-token decoding.

OFFICIAL / DEEP REFERENCE

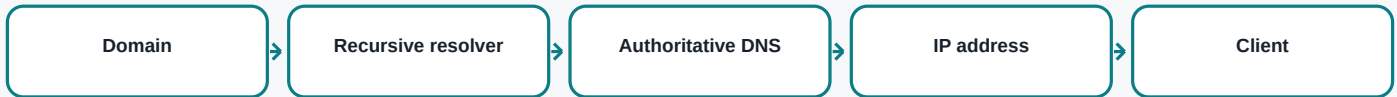
Prompt Engineering - Lilian Weng

Surveys prompting techniques such as few-shot prompting, decomposition, reasoning, self-consistency, and ReAct.

DNS and name resolution

WHAT TO LEARN - IN ORDER

- Understand how names become routable destinations and why caching makes DNS both fast and occasionally stale.



LINEAR LEARNING PATH

1. Recursive versus authoritative resolution
2. A/AAAA/CNAME records
3. TTL and resolver caches
4. Failure modes during record changes

DO THIS

Use dig/nslookup and trace the records for one domain you own or know.

MID-SENIOR CHECKPOINT

What happens during a deployment if clients keep an old DNS answer longer than expected?

REQUIRED READING

What is DNS? - Cloudflare

Explains recursive resolution, authoritative servers, DNS records, and why name resolution precedes application traffic.

OFFICIAL / DEEP REFERENCE

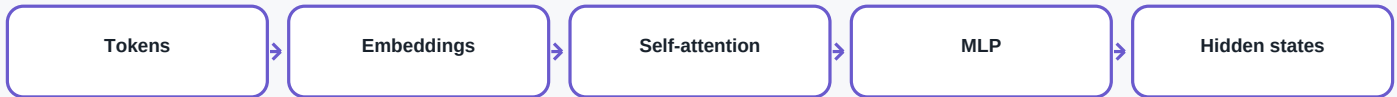
DNS Concepts - RFC 1034

Original DNS concepts and architecture reference.

Transformer intuition: embeddings, attention, MLP

WHAT TO LEARN - IN ORDER

- Build the minimum internal model needed to understand context, attention, KV cache, and inference behavior later.



LINEAR LEARNING PATH

1. Map token IDs to embeddings
2. See self-attention as content-dependent mixing
3. Understand the role of the feed-forward/MLP block
4. Follow residual connections through layers

DO THIS

Walk one short sentence through an annotated Transformer implementation and label tensor shapes.

MID-SENIOR CHECKPOINT

Which parts of the Transformer depend on sequence length and which depend mostly on model width?

REQUIRED READING

[The Annotated Transformer - Harvard NLP](#)

Walks through a Transformer implementation and connects the paper to working code.

OFFICIAL / DEEP REFERENCE

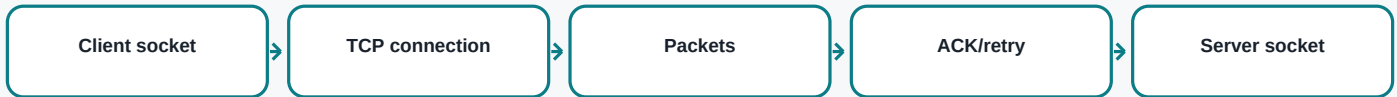
[Attention Is All You Need - Vaswani et al.](#)

The canonical Transformer paper: self-attention, multi-head attention, residual paths, and positional encoding.

TCP and reliable transport

WHAT TO LEARN - IN ORDER

- Understand how reliable byte streams are created over unreliable networks and why application latency includes transport behavior.



LINEAR LEARNING PATH

1. Connection state and sequence numbers
2. ACKs and retransmission
3. Flow/congestion control intuition
4. Connection setup and failure

DO THIS

Capture a TCP connection with a packet analyzer and identify SYN, ACK, data, and FIN/RST.

MID-SENIOR CHECKPOINT

Why can packet loss increase application latency even when servers are healthy?

REQUIRED READING

TCP Specification - RFC 9293

Canonical reference for reliable byte streams, connection state, sequence numbers, retransmission, and flow control.

OFFICIAL / DEEP REFERENCE

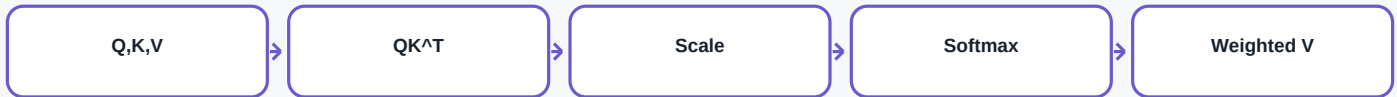
Overview of HTTP - MDN

Explains methods, headers, responses, intermediaries, caching, and stateless request semantics.

Attention mathematically

WHAT TO LEARN - IN ORDER

- Move from intuition to the actual attention equation and interpret every term.



LINEAR LEARNING PATH

1. Construct Q, K, and V
2. Compute scaled dot products
3. Apply softmax weights
4. Explain why multiple heads can learn different relationships

DO THIS

Calculate attention for 3 toy tokens with 2-dimensional vectors by hand or in Python.

MID-SENIOR CHECKPOINT

What does changing one key vector do to every query that attends to it?

REQUIRED READING

Attention Is All You Need - Vaswani et al.

The canonical Transformer paper: self-attention, multi-head attention, residual paths, and positional encoding.

OFFICIAL / DEEP REFERENCE

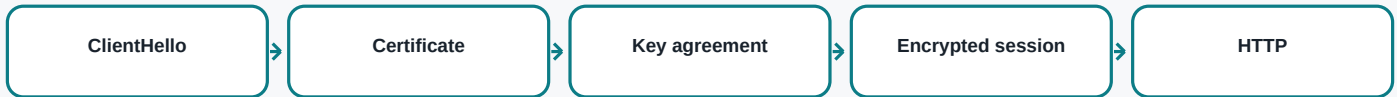
The Annotated Transformer - Harvard NLP

Walks through a Transformer implementation and connects the paper to working code.

TLS and secure sessions

WHAT TO LEARN - IN ORDER

- Understand where identity, encryption, and key agreement enter the request path.



LINEAR LEARNING PATH

1. ClientHello/ServerHello intuition
2. Certificate verification
3. Key agreement
4. Session resumption and latency

DO THIS

Open browser certificate details for a site and identify issuer, subject, validity, and SANs.

MID-SENIOR CHECKPOINT

What fails when a certificate is valid cryptographically but issued for the wrong hostname?

REQUIRED READING

What Happens in a TLS Handshake? - Cloudflare

Explains certificates, key agreement, identity verification, and how secure sessions are established.

OFFICIAL / DEEP REFERENCE

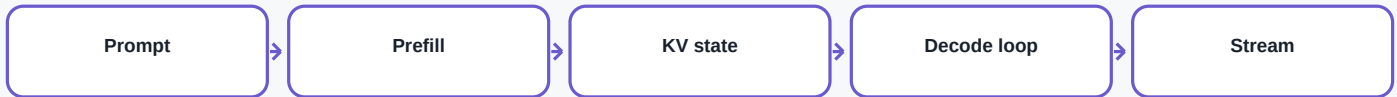
Overview of HTTP - MDN

Explains methods, headers, responses, intermediaries, caching, and stateless request semantics.

Inference lifecycle: prefill vs decode

WHAT TO LEARN - IN ORDER

- Learn why processing the prompt and generating tokens are different computational phases.



LINEAR LEARNING PATH

1. Prefill processes prompt tokens in parallel
2. Decode generates tokens autoregressively
3. TTFT differs from inter-token latency
4. Separate model latency from network/tool latency

DO THIS

Measure time-to-first-token and total response time for short vs long outputs.

MID-SENIOR CHECKPOINT

Why can halving output length reduce latency more than halving prompt length?

REQUIRED READING

Latency Optimization - OpenAI

Explains how model choice, token count, request count, parallelism, and UI design affect end-to-end latency.

OFFICIAL / DEEP REFERENCE

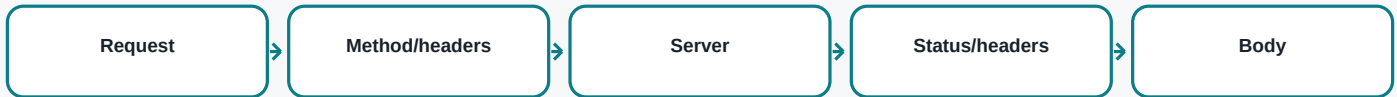
Production Best Practices - OpenAI

Production checklist for reliability, latency, usage limits, and operational safeguards.

HTTP semantics

WHAT TO LEARN - IN ORDER

- Learn the protocol contract applications depend on: method, URI, headers, status, caching, and statelessness.



LINEAR LEARNING PATH

1. GET/POST/PUT/PATCH/DELETE semantics
2. Status-code classes
3. Headers and representation metadata
4. Stateless requests and intermediaries

DO THIS

Inspect request/response headers with curl and explain every important field.

MID-SENIOR CHECKPOINT

Why does "stateless HTTP" not mean your application has no state?

REQUIRED READING

Overview of HTTP - MDN

Explains methods, headers, responses, intermediaries, caching, and stateless request semantics.

OFFICIAL / DEEP REFERENCE

APIs as Infrastructure: Stripe API Versioning

Shows how to evolve a public API without silently breaking old clients.

KV cache and inference memory

WHAT TO LEARN - IN ORDER

- Understand why active context becomes a memory resource and why LLM server capacity is not simply requests/second.



LINEAR LEARNING PATH

1. Keys/values can be reused during decode
2. KV memory scales with active tokens
3. Long contexts reduce concurrent capacity
4. Memory fragmentation motivates paged allocation

DO THIS

Estimate KV-cache memory for a hypothetical model and 8k-token sequence.

MID-SENIOR CHECKPOINT

Why can a 24GB GPU serve many short conversations but only a few very long ones?

REQUIRED READING

[vLLM: PagedAttention and High-throughput LLM Serving](#)

Explains KV-cache memory pressure, PagedAttention, and why serving throughput is a scheduling and memory problem.

OFFICIAL / DEEP REFERENCE

[vLLM - Open-source LLM inference and serving engine](#)

Implementation reference for continuous batching, paged KV cache, OpenAI-compatible serving, and benchmarks.

API contracts and versioning

WHAT TO LEARN - IN ORDER

- Treat APIs as long-lived compatibility contracts rather than convenient function calls.



LINEAR LEARNING PATH

1. Resource and endpoint design
2. Backward compatibility
3. Idempotency and error contracts
4. Versioning and deprecation strategy

DO THIS

Design v1 and v2 of a /payments endpoint while keeping one old client working.

MID-SENIOR CHECKPOINT

What is the migration cost of changing the meaning of a field without changing its name?

REQUIRED READING

APIs as Infrastructure: Stripe API Versioning

Shows how to evolve a public API without silently breaking old clients.

OFFICIAL / DEEP REFERENCE

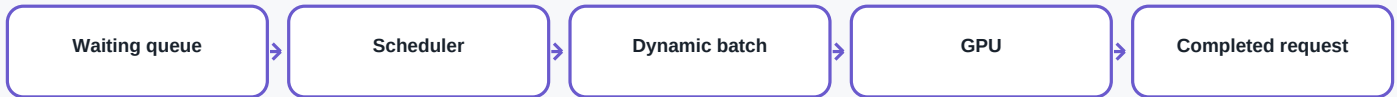
Idempotent Requests - Stripe

Shows how API clients safely retry write operations using stable idempotency keys.

Continuous batching and serving throughput

WHAT TO LEARN - IN ORDER

- Learn how inference schedulers mix active sequences rather than waiting for whole static batches to finish.



LINEAR LEARNING PATH

1. Static batching wastes slots on finished requests
2. Continuous batching replaces completed sequences
3. Prompt/decode work have different costs
4. Throughput depends on scheduler and memory pressure

DO THIS

Run a serving benchmark with concurrency 1, 4, 16, and 64 and record tokens/sec and p95 latency.

MID-SENIOR CHECKPOINT

Why can throughput rise while p99 latency becomes unacceptable?

REQUIRED READING

[vLLM - Open-source LLM inference and serving engine](#)

Implementation reference for continuous batching, paged KV cache, OpenAI-compatible serving, and benchmarks.

OFFICIAL / DEEP REFERENCE

[vLLM: PagedAttention and High-throughput LLM Serving](#)

Explains KV-cache memory pressure, PagedAttention, and why serving throughput is a scheduling and memory problem.

Back-of-envelope capacity estimation

WHAT TO LEARN - IN ORDER

- Turn product scale into rough QPS, storage, bandwidth, and worker requirements before choosing technologies.



LINEAR LEARNING PATH

1. Average vs peak QPS
2. Read/write ratio
3. Storage growth per day/year
4. Bandwidth and concurrency estimates

DO THIS

Estimate a 10M-user product: DAU, requests/day, peak QPS, annual storage, and egress.

MID-SENIOR CHECKPOINT

Which estimate is most sensitive to a 10x error and why?

REQUIRED READING

Google Cloud Architecture Framework

High-level framework for capacity, reliability, performance, operations, and architecture trade-offs.

OFFICIAL / DEEP REFERENCE

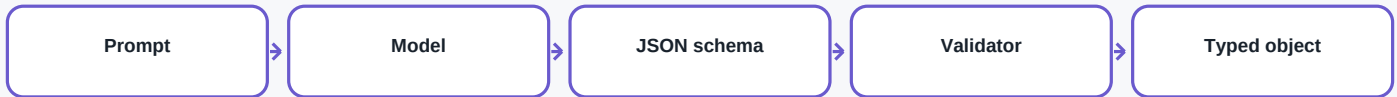
AWS Architecture Center

Broad reference collection for real cloud architectures, patterns, scaling, and operational trade-offs.

Structured outputs and typed AI interfaces

WHAT TO LEARN - IN ORDER

- Replace brittle string parsing with schemas the application can validate and reason about.



LINEAR LEARNING PATH

1. Define output schemas
2. Use enums and required fields
3. Validate and reject malformed outputs
4. Keep deterministic business rules outside prose generation

DO THIS

Wrap one extraction task in a Pydantic/JSON schema and test 20 inputs.

MID-SENIOR CHECKPOINT

When should validation fail closed instead of asking the model to repair itself?

REQUIRED READING

Structured Outputs - OpenAI

Shows how to turn probabilistic text generation into schema-constrained program interfaces.

OFFICIAL / DEEP REFERENCE

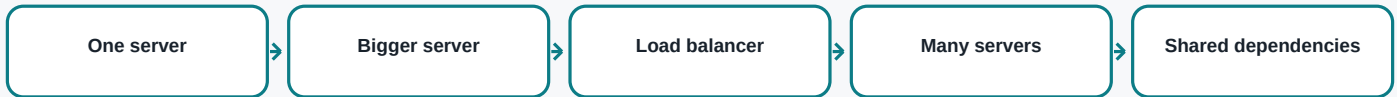
Pydantic Documentation

Implementation reference for typed validation and schema-driven application contracts.

Vertical vs horizontal scaling

WHAT TO LEARN - IN ORDER

- Understand why scale-up is simple but bounded, while scale-out requires statelessness, routing, and coordination.



LINEAR LEARNING PATH

1. Scale-up vs scale-out
2. Failure domains
3. Shared state challenges
4. When one large machine is still the right answer

DO THIS

Take a single-server web app and sketch the changes required to run 5 replicas.

MID-SENIOR CHECKPOINT

What hidden local state prevents an application from scaling horizontally?

REQUIRED READING

Google Cloud Architecture Framework

High-level framework for capacity, reliability, performance, operations, and architecture trade-offs.

OFFICIAL / DEEP REFERENCE

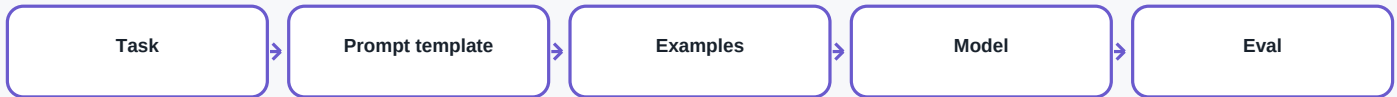
HTTP Load Balancing - NGINX

Official reference for round-robin, least-connections, IP-hash routing, and balancing across backend servers.

Prompt engineering as software engineering

WHAT TO LEARN - IN ORDER

- Treat prompts as versioned program artifacts with tests, examples, and explicit contracts.



LINEAR LEARNING PATH

1. Instruction hierarchy
2. Few-shot examples
3. Decomposition and constraints
4. Prompt regression tests

DO THIS

Create a prompt test suite with 20 cases, then intentionally break one instruction and observe failures.

MID-SENIOR CHECKPOINT

Why is "the prompt looked better" not a valid release criterion?

REQUIRED READING

Prompt Engineering - Lilian Weng

Surveys prompting techniques such as few-shot prompting, decomposition, reasoning, self-consistency, and ReAct.

OFFICIAL / DEEP REFERENCE

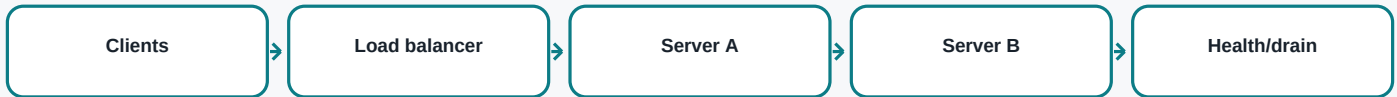
Prompting Guide - OpenAI

Current prompt construction and prompt-test guidance.

Load balancing and health checks

WHAT TO LEARN - IN ORDER

- Learn how a fleet becomes one logical service and why routing policy changes tail latency and failure behavior.



LINEAR LEARNING PATH

1. Round robin vs least connections
2. Health checks
3. Connection draining
4. Affinity and its trade-offs

DO THIS

Put 3 local HTTP servers behind NGINX and kill one while sending traffic.

MID-SENIOR CHECKPOINT

When does sticky routing improve locality but hurt resilience?

REQUIRED READING

HTTP Load Balancing - NGINX

Official reference for round-robin, least-connections, IP-hash routing, and balancing across backend servers.

OFFICIAL / DEEP REFERENCE

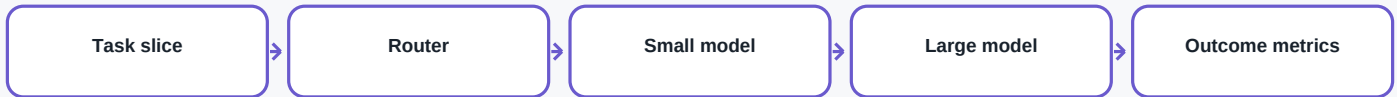
NGINX Request Limiting

Shows how front-door routing layers can also enforce admission limits.

Model selection: quality, latency, cost

WHAT TO LEARN - IN ORDER

- Choose the smallest/fastest model that satisfies a measured workload rather than defaulting to one model for everything.



LINEAR LEARNING PATH

1. Define workload slices
2. Compare quality per slice
3. Measure latency and cost
4. Route by difficulty or risk

DO THIS

Benchmark 2 models on 30 representative tasks and plot quality vs cost.

MID-SENIOR CHECKPOINT

When is a more expensive model actually cheaper at the product level?

REQUIRED READING

Model Selection Guide - OpenAI

Useful for thinking about workload-model fit rather than defaulting to the largest model.

OFFICIAL / DEEP REFERENCE

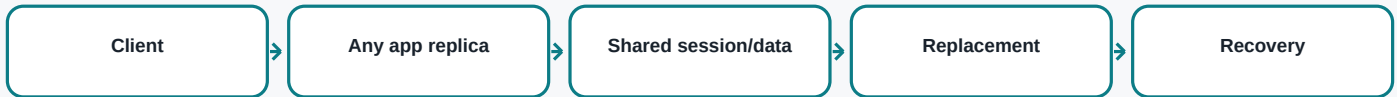
API Deployment Checklist - OpenAI

Current deployment checklist spanning model choice, reasoning budgets, tools, compaction, caching, and reliability.

Stateless services and session design

WHAT TO LEARN - IN ORDER

- Learn why local mutable state creates scaling and failover problems.



LINEAR LEARNING PATH

1. Stateless process principle
2. External session stores
3. Sticky sessions vs portable sessions
4. Replacement and autoscaling

DO THIS

Move an in-memory login session to Redis or a signed token and then run two app replicas.

MID-SENIOR CHECKPOINT

Which state truly belongs to the process, and which must survive process death?

REQUIRED READING

The Twelve-Factor App: Processes

Explains why stateless share-nothing application processes make horizontal scaling and replacement easier.

OFFICIAL / DEEP REFERENCE

Redis Documentation

Reference for in-memory data structures, caching, persistence, replication, clustering, and operational behavior.

Embeddings and semantic representation

WHAT TO LEARN - IN ORDER

- Understand what embeddings encode, what distance means, and where semantic similarity is useful or misleading.



LINEAR LEARNING PATH

1. Embedding vector intuition
2. Cosine/dot-product similarity
3. Semantic search and clustering
4. Domain/model mismatch

DO THIS

Embed 50 sentences and inspect nearest neighbors for 5 queries.

MID-SENIOR CHECKPOINT

Why does semantic similarity not guarantee factual relevance?

REQUIRED READING

Vector Embeddings - OpenAI

Introduces embeddings, similarity, and common semantic-search use cases.

OFFICIAL / DEEP REFERENCE

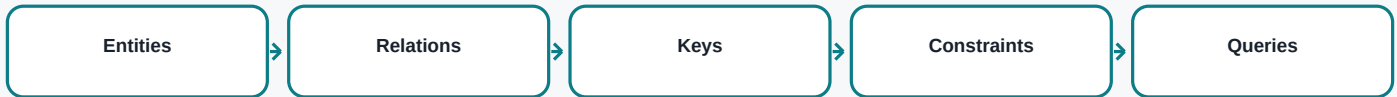
Sentence Transformers Documentation

Practical reference for bi-encoders, semantic similarity, retrieval, and embedding evaluation.

Relational data modeling and constraints

WHAT TO LEARN - IN ORDER

- Model invariants in the database so correctness does not depend entirely on application code.



LINEAR LEARNING PATH

1. Primary and foreign keys
2. Unique/check constraints
3. Normalization intuition
4. Transaction boundaries around invariants

DO THIS

Design users, organizations, projects, and memberships with keys and constraints.

MID-SENIOR CHECKPOINT

Which invariant should be enforced by the database rather than a service check?

REQUIRED READING

PostgreSQL Constraints

Official reference for primary keys, foreign keys, unique constraints, and enforcing relational invariants.

OFFICIAL / DEEP REFERENCE

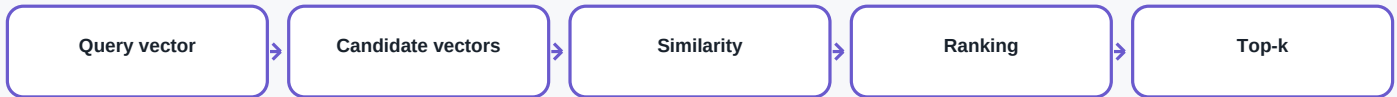
PostgreSQL Transactions Tutorial

Canonical introduction to BEGIN, COMMIT, ROLLBACK, atomic multi-step operations, and savepoints.

Cosine similarity and vector retrieval

WHAT TO LEARN - IN ORDER

- Move from "embeddings are vectors" to the actual similarity calculation used by retrievers.



LINEAR LEARNING PATH

1. Normalize vectors
2. Compute cosine similarity
3. Understand dot product vs cosine
4. Use thresholds carefully

DO THIS

Compute similarity for 5 toy vectors by hand and verify with code.

MID-SENIOR CHECKPOINT

Why can a fixed similarity threshold behave differently across embedding models?

REQUIRED READING

Vector Embeddings - OpenAI

Introduces embeddings, similarity, and common semantic-search use cases.

OFFICIAL / DEEP REFERENCE

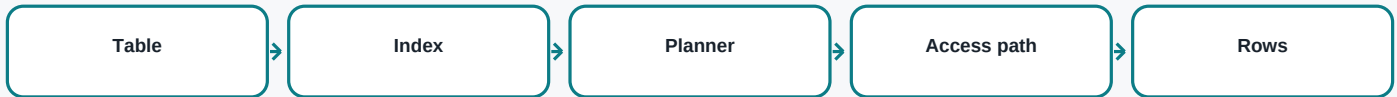
Sentence Transformers Documentation

Practical reference for bi-encoders, semantic similarity, retrieval, and embedding evaluation.

Indexes and query access paths

WHAT TO LEARN - IN ORDER

- Understand why an index accelerates some queries, slows writes, and can still be ignored by the planner.



LINEAR LEARNING PATH

1. B-tree mental model
2. Selectivity
3. Composite index ordering
4. EXPLAIN and query planner choices

DO THIS

Create a 1M-row table, benchmark an indexed lookup before/after the index, then inspect EXPLAIN.

MID-SENIOR CHECKPOINT

Why is "this column is indexed" not proof that the query is efficient?

REQUIRED READING

PostgreSQL Index Types

Explains B-tree, hash, GiST, GIN, SP-GiST, and BRIN indexes and the access patterns they support.

OFFICIAL / DEEP REFERENCE

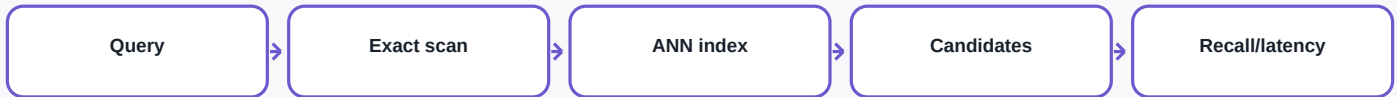
Using EXPLAIN - PostgreSQL

Official guide to understanding query plans and index usage.

Exact vs approximate nearest-neighbor search

WHAT TO LEARN - IN ORDER

- Understand when scanning every vector stops being practical and what ANN gives up for speed.



LINEAR LEARNING PATH

1. Flat/exact search
2. Recall vs latency
3. Candidate pruning
4. Memory/index-build trade-offs

DO THIS

Compare FAISS IndexFlat with one ANN index on the same 100k vectors.

MID-SENIOR CHECKPOINT

What recall loss is acceptable when retrieved evidence feeds an LLM?

REQUIRED READING

[FAISS Index Guide](#)

Explains exact versus approximate search and how to choose indexes based on scale, memory, and recall.

OFFICIAL / DEEP REFERENCE

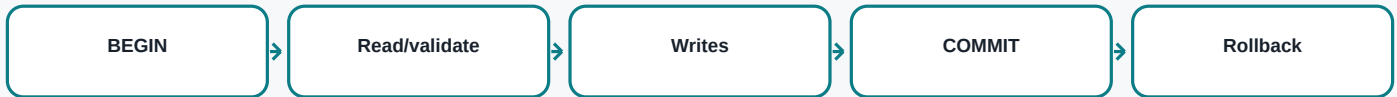
[pgvector - Vector Similarity Search for Postgres](#)

Concrete implementation reference for exact search, HNSW, IVFFlat, filtering, and SQL integration.

Transactions and ACID

WHAT TO LEARN - IN ORDER

- Treat a transaction as a correctness boundary for multiple dependent writes.



LINEAR LEARNING PATH

1. Atomicity
2. Consistency as application invariants
3. Isolation
4. Durability

DO THIS

Implement a money transfer and force an exception between debit and credit.

MID-SENIOR CHECKPOINT

Which partial states must be impossible even if the process crashes?

REQUIRED READING

PostgreSQL Transactions Tutorial

Canonical introduction to BEGIN, COMMIT, ROLLBACK, atomic multi-step operations, and savepoints.

OFFICIAL / DEEP REFERENCE

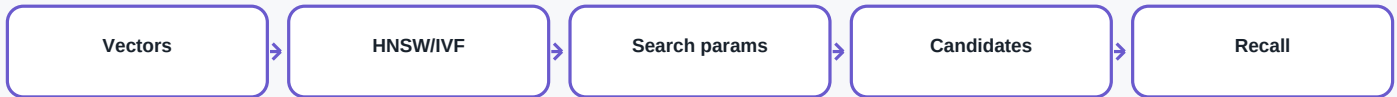
PostgreSQL Transaction Isolation

Explains isolation levels, anomalies, snapshots, serialization failures, and concurrency semantics.

HNSW and IVF internally

WHAT TO LEARN - IN ORDER

- Learn two common ANN families well enough to reason about memory, build cost, and query-time tuning.



LINEAR LEARNING PATH

1. HNSW graph navigation
2. IVF coarse partitioning
3. Search breadth/probe controls
4. Recall-memory-latency trade-offs

DO THIS

Build HNSW and IVFFlat indexes with pgvector or FAISS and vary search parameters.

MID-SENIOR CHECKPOINT

Why can a faster ANN query still produce a worse end-to-end RAG product?

REQUIRED READING

[pgvector - Vector Similarity Search for Postgres](#)

Concrete implementation reference for exact search, HNSW, IVFFlat, filtering, and SQL integration.

OFFICIAL / DEEP REFERENCE

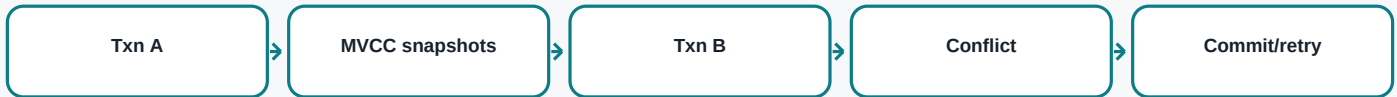
[FAISS Index Guide](#)

Explains exact versus approximate search and how to choose indexes based on scale, memory, and recall.

MVCC and transaction isolation

WHAT TO LEARN - IN ORDER

- Understand how concurrent transactions see snapshots and where anomalies still occur.



LINEAR LEARNING PATH

1. Read Committed
2. Repeatable Read
3. Serializable
4. Serialization retry

DO THIS

Run two database sessions and reproduce a non-repeatable read or write conflict.

MID-SENIOR CHECKPOINT

Which isolation level preserves your business invariant with the least coordination?

REQUIRED READING

PostgreSQL Transaction Isolation

Explains isolation levels, anomalies, snapshots, serialization failures, and concurrency semantics.

OFFICIAL / DEEP REFERENCE

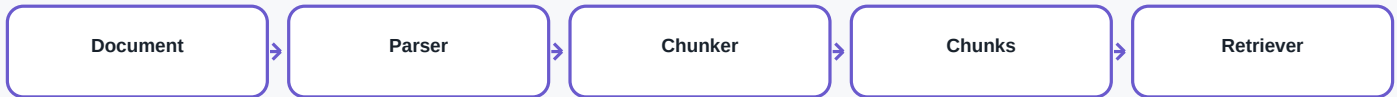
PostgreSQL Transactions Tutorial

Canonical introduction to BEGIN, COMMIT, ROLLBACK, atomic multi-step operations, and savepoints.

Chunking strategies

WHAT TO LEARN - IN ORDER

- Chunking is an information-architecture decision: it determines what the retriever can ever return.



LINEAR LEARNING PATH

1. Fixed-size chunks
2. Sentence/section boundaries
3. Overlap
4. Hierarchical/semantic chunking

DO THIS

Chunk one PDF three ways and compare retrieval results on 10 questions.

MID-SENIOR CHECKPOINT

When does larger context improve recall but reduce precision?

REQUIRED READING

Text Splitters - LangChain Concepts

Practical taxonomy of chunking and splitting approaches used before retrieval.

OFFICIAL / DEEP REFERENCE

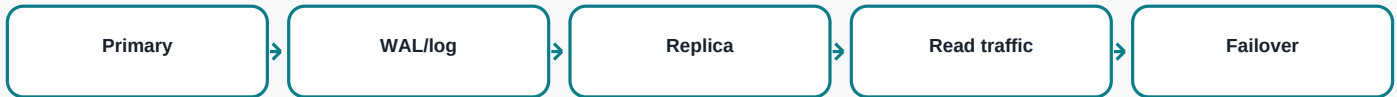
Contextual Retrieval - Anthropic

Shows why isolated chunks lose context and how contextualizing chunks can improve retrieval.

Replication and failover

WHAT TO LEARN - IN ORDER

- Learn how copies improve read scale and availability while introducing lag and failover complexity.



LINEAR LEARNING PATH

1. Primary/replica roles
2. Synchronous vs asynchronous replication
3. Replication lag
4. Failover and promotion

DO THIS

Run a local primary/replica setup or inspect replication metrics in a managed DB.

MID-SENIOR CHECKPOINT

*Can a successful write be missing after failover?
Under what replication policy?*

REQUIRED READING

PostgreSQL High Availability and Replication

Official overview of replication and availability patterns for PostgreSQL systems.

OFFICIAL / DEEP REFERENCE

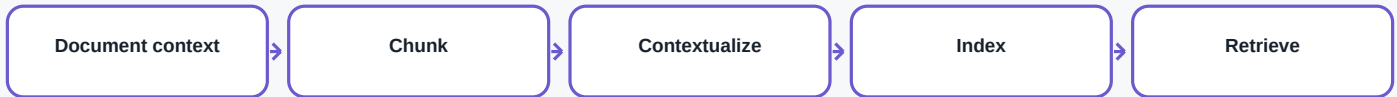
PostgreSQL Transaction Isolation

Explains isolation levels, anomalies, snapshots, serialization failures, and concurrency semantics.

Contextual retrieval

WHAT TO LEARN - IN ORDER

- Preserve the meaning a chunk had in its original document rather than embedding isolated fragments blindly.



LINEAR LEARNING PATH

1. Lost context problem
2. Contextualized chunks
3. Lexical + dense retrieval
4. Measure retrieval change rather than assume improvement

DO THIS

Prefix chunks with section/document context and compare Recall@k against raw chunks.

MID-SENIOR CHECKPOINT

When can adding context to every chunk create noise instead of signal?

REQUIRED READING

Contextual Retrieval - Anthropic

Shows why isolated chunks lose context and how contextualizing chunks can improve retrieval.

OFFICIAL / DEEP REFERENCE

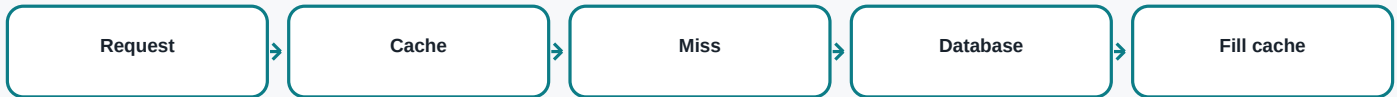
Patterns for Building LLM-based Systems - Eugene Yan

Connects retrieval, routing, generation, validation, caching, and evaluation into reusable system patterns.

Cache-aside, TTL and invalidation

WHAT TO LEARN - IN ORDER

- Treat caching as a correctness and load-management problem, not just a speed trick.



LINEAR LEARNING PATH

1. Cache-aside flow
2. TTL
3. Invalidation on writes
4. Cache failure fallback

DO THIS

Add Redis to one read-heavy endpoint and measure DB QPS and p95 latency.

MID-SENIOR CHECKPOINT

What happens to the database when the entire cache expires at once?

REQUIRED READING

Redis Documentation

Reference for in-memory data structures, caching, persistence, replication, clustering, and operational behavior.

OFFICIAL / DEEP REFERENCE

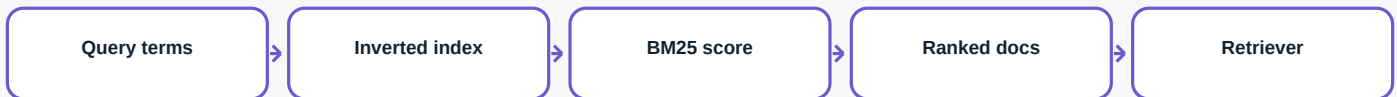
Redis Persistence

Explains what Redis does and does not guarantee when used beyond disposable caching.

Sparse retrieval and BM25

WHAT TO LEARN - IN ORDER

- Understand why exact terms, identifiers, names, and rare tokens often require lexical retrieval alongside embeddings.



LINEAR LEARNING PATH

1. Term frequency
2. Inverse document frequency intuition
3. BM25 scoring
4. Failure cases for dense-only search

DO THIS

Index technical docs and test queries containing error codes, product IDs, and exact names.

MID-SENIOR CHECKPOINT

Which query classes should bypass or heavily weight lexical search?

REQUIRED READING

BM25 Similarity - Elasticsearch

Official implementation reference for lexical ranking and BM25-style scoring.

OFFICIAL / DEEP REFERENCE

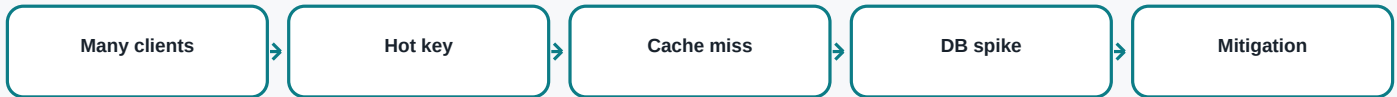
Patterns for Building LLM-based Systems - Eugene Yan

Connects retrieval, routing, generation, validation, caching, and evaluation into reusable system patterns.

Cache stampede and hot keys

WHAT TO LEARN - IN ORDER

- Learn why a cache can protect a database during normal operation and overload it during coordinated misses.



LINEAR LEARNING PATH

1. Thundering herd
2. Request coalescing
3. TTL jitter
4. Hot-key replication/local caches

DO THIS

Expire one popular cached key while generating high concurrency and measure DB fan-in.

MID-SENIOR CHECKPOINT

How do you keep one popular key from becoming one machine's capacity limit?

REQUIRED READING

Redis Key Eviction

Explains memory limits and eviction policies such as LRU and LFU.

OFFICIAL / DEEP REFERENCE

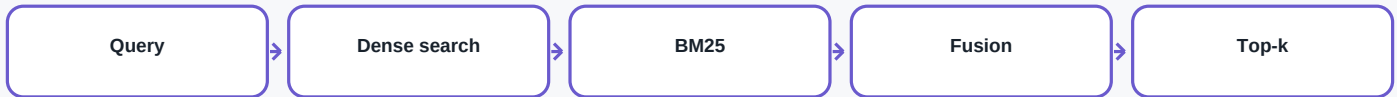
Redis Documentation

Reference for in-memory data structures, caching, persistence, replication, clustering, and operational behavior.

Hybrid search

WHAT TO LEARN - IN ORDER

- Combine semantic and lexical retrieval because their failure modes are complementary.



LINEAR LEARNING PATH

1. Dense retrieval strengths
2. BM25 strengths
3. Score fusion/rank fusion
4. Metadata filtering

DO THIS

Build a two-stage hybrid retriever and compare against dense-only and lexical-only baselines.

MID-SENIOR CHECKPOINT

How do you normalize scores from two retrieval systems with different scales?

REQUIRED READING

Patterns for Building LLM-based Systems - Eugene Yan

Connects retrieval, routing, generation, validation, caching, and evaluation into reusable system patterns.

OFFICIAL / DEEP REFERENCE

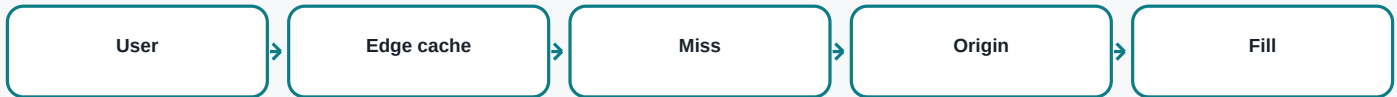
BM25 Similarity - Elasticsearch

Official implementation reference for lexical ranking and BM25-style scoring.

CDN and edge caching

WHAT TO LEARN - IN ORDER

- Understand how geographically distributed caches remove origin work and network distance from common requests.



LINEAR LEARNING PATH

1. Edge POP
2. Cache key
3. Hit/miss
4. Invalidation and stale content

DO THIS

Serve a static asset through a CDN and inspect Age/Cache-Control headers across repeated requests.

MID-SENIOR CHECKPOINT

When is stale-but-fast content better than origin-consistent content?

REQUIRED READING

What is a CDN? - Cloudflare

Explains edge caching, origin offload, geographic delivery, and cache-hit behavior.

OFFICIAL / DEEP REFERENCE

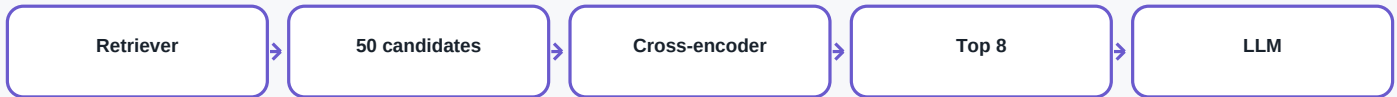
HTTP Caching - MDN

Reference for Cache-Control, validators, freshness, and shared/private caches.

Reranking

WHAT TO LEARN - IN ORDER

- Use a stronger pairwise model after high-recall retrieval to improve the precision of context sent to the LLM.



LINEAR LEARNING PATH

1. First-stage retrieval
2. Cross-encoder scoring
3. Top-N to top-k reduction
4. Latency/quality trade-off

DO THIS

Retrieve 50 candidates, rerank them, and compare nDCG or answer accuracy.

MID-SENIOR CHECKPOINT

How much reranker latency is justified by a small precision gain?

REQUIRED READING

Cross-Encoder Reranking - Sentence Transformers

Shows how rerankers trade extra compute for higher precision after first-stage retrieval.

OFFICIAL / DEEP REFERENCE

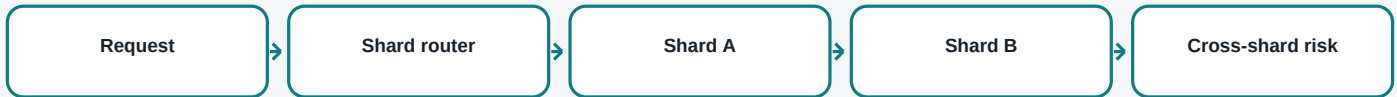
Patterns for Building LLM-based Systems - Eugene Yan

Connects retrieval, routing, generation, validation, caching, and evaluation into reusable system patterns.

Database partitioning and sharding

WHAT TO LEARN - IN ORDER

- Split data by ownership boundaries while accounting for cross-shard joins, transactions, and hot partitions.



LINEAR LEARNING PATH

1. Choose shard key
2. Routing layer
3. Cross-shard operations
4. Online migration

DO THIS

Shard a toy dataset by `tenant_id` and list every query that becomes cross-shard.

MID-SENIOR CHECKPOINT

Which key gives locality without concentrating the busiest tenants?

REQUIRED READING

Partitioning GitHub Relational Databases

Real migration showing how a large relational system moves toward partitioned database clusters.

OFFICIAL / DEEP REFERENCE

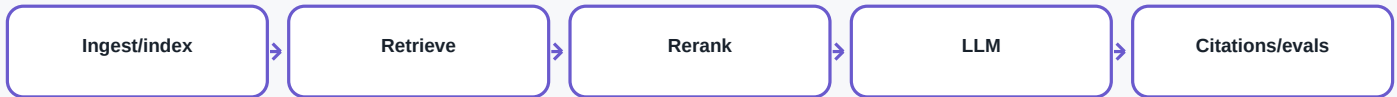
Eliminating Cold Starts 2: Shard and Conquer - Cloudflare

Production example of consistent hashing, stable ownership, locality, and remapping under node churn.

Production RAG and retrieval evaluation

WHAT TO LEARN - IN ORDER

- Join ingestion, indexing, search, reranking, generation, citations, permissions, and evals into one measured system.



LINEAR LEARNING PATH

1. Evaluate retrieval separately from generation
2. Track freshness and permissions
3. Measure Recall@k/MRR/nDCG
4. Verify claims against retrieved evidence

DO THIS

Create a 30-question retrieval eval set and compare at least two pipelines.

MID-SENIOR CHECKPOINT

If answer quality is low, how do you prove whether retrieval or generation is responsible?

REQUIRED READING

Contextual Retrieval - Anthropic

Shows why isolated chunks lose context and how contextualizing chunks can improve retrieval.

OFFICIAL / DEEP REFERENCE

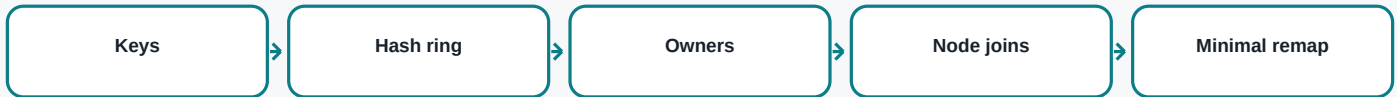
Evaluation of Ranked Retrieval - Stanford IR Book

Classic reference for precision, recall, MAP, and ranked-retrieval evaluation.

Consistent hashing and virtual nodes

WHAT TO LEARN - IN ORDER

- Preserve ownership locality while minimizing remapping when a distributed fleet changes membership.



LINEAR LEARNING PATH

1. Hash ring mental model
2. Virtual nodes
3. Node join/leave
4. Hot-key limitations

DO THIS

Simulate 10k keys on 10 nodes with modulo hashing vs consistent hashing, then add one node.

MID-SENIOR CHECKPOINT

Why does balanced key count not guarantee balanced traffic?

REQUIRED READING

Eliminating Cold Starts 2: Shard and Conquer - Cloudflare

Production example of consistent hashing, stable ownership, locality, and remapping under node churn.

OFFICIAL / DEEP REFERENCE

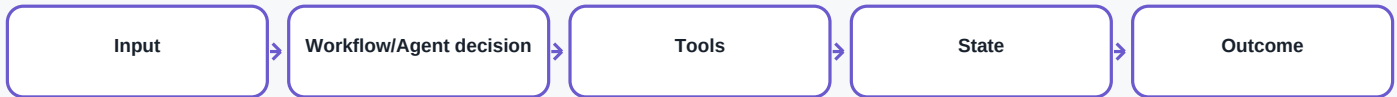
Dynamo: Amazon Highly Available Key-value Store

Classic system covering consistent hashing, quorums, eventual consistency, vector clocks, and always-on availability.

Workflow vs agent

WHAT TO LEARN - IN ORDER

- Use deterministic workflows when the path is known; use agents only where adaptive decision-making is valuable.



LINEAR LEARNING PATH

1. Workflow: predefined control flow
2. Agent: model chooses next action
3. Escalate complexity gradually
4. Keep deterministic steps deterministic

DO THIS

Take one multi-step AI feature and write both a workflow and agent design.

MID-SENIOR CHECKPOINT

What evidence justifies giving control-flow authority to the model?

REQUIRED READING

Building Effective Agents - Anthropic

Defines workflows versus agents and argues for simple composable patterns before complex frameworks.

OFFICIAL / DEEP REFERENCE

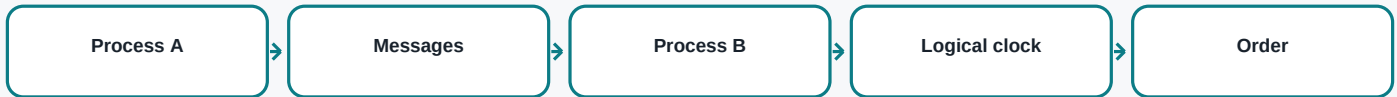
A Practical Guide to Building AI Agents - OpenAI

Production-oriented overview of when to use agents, how to orchestrate them, and how to apply safeguards.

Time and ordering in distributed systems

WHAT TO LEARN - IN ORDER

- Understand why wall-clock timestamps alone cannot reliably express causal ordering across machines.



LINEAR LEARNING PATH

1. Clock skew
2. Events and happens-before
3. Logical time
4. Ordering vs real time

DO THIS

Simulate two processes with unsynchronized clocks and generate an impossible-looking timestamp order.

MID-SENIOR CHECKPOINT

Which decisions require causal ordering rather than physical time?

REQUIRED READING

Time, Clocks, and the Ordering of Events - Lamport

Foundational paper for reasoning about event ordering and causality without relying on synchronized wall clocks.

OFFICIAL / DEEP REFERENCE

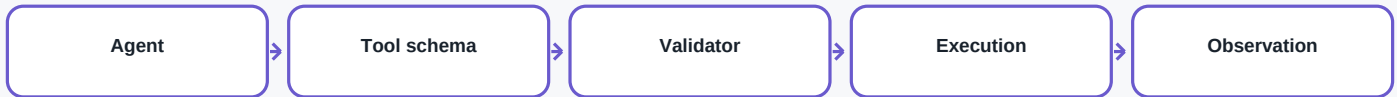
Dapper: Large-Scale Distributed Tracing Infrastructure - Google

Foundational tracing paper for reconstructing one request across many distributed services.

Tool and API design for agents

WHAT TO LEARN - IN ORDER

- A tool is an interface contract the model must discover, select, populate, and interpret correctly.



LINEAR LEARNING PATH

1. Clear tool boundaries
2. Typed arguments
3. Compact useful outputs
4. Action-specific errors

DO THIS

Redesign 5 existing API endpoints into agent tools and remove ambiguous parameters.

MID-SENIOR CHECKPOINT

Which tool mistakes should be impossible by schema rather than discouraged by prompt?

REQUIRED READING

Writing Effective Tools for Agents - Anthropic

Shows how tool naming, boundaries, output design, and evals affect agent performance.

OFFICIAL / DEEP REFERENCE

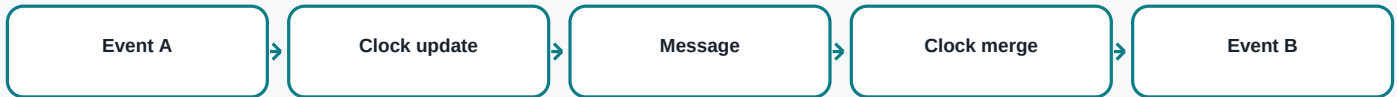
Structured Outputs - OpenAI

Shows how to turn probabilistic text generation into schema-constrained program interfaces.

Causality and Lamport clocks

WHAT TO LEARN - IN ORDER

- Learn the happens-before relation and what logical clocks can and cannot prove.



LINEAR LEARNING PATH

1. Local increment
2. Message send/receive update
3. Causal order
4. Concurrent events

DO THIS

Implement a Lamport clock for two simulated nodes exchanging messages.

MID-SENIOR CHECKPOINT

Why does $C(a) < C(b)$ not necessarily prove that a caused b ?

REQUIRED READING

Time, Clocks, and the Ordering of Events - Lamport

Foundational paper for reasoning about event ordering and causality without relying on synchronized wall clocks.

OFFICIAL / DEEP REFERENCE

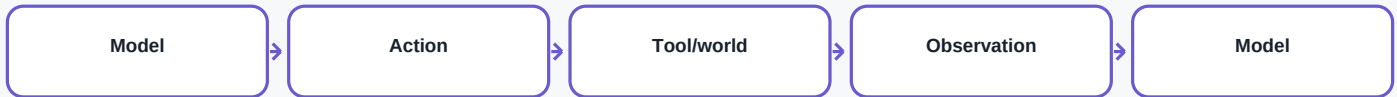
Dynamo: Amazon Highly Available Key-value Store

Classic system covering consistent hashing, quorums, eventual consistency, vector clocks, and always-on availability.

ReAct and the agent execution loop

WHAT TO LEARN - IN ORDER

- Understand the simplest useful agent loop: decide, act, observe, update state, and stop.



LINEAR LEARNING PATH

1. Reason/action alternation
2. Tool observations change future decisions
3. Explicit stop conditions
4. Bound steps and costs

DO THIS

Implement a 3-tool ReAct loop with `MAX_STEPS=8` and trace every decision.

MID-SENIOR CHECKPOINT

How do you tell useful adaptation from a runaway loop?

REQUIRED READING

ReAct: Synergizing Reasoning and Acting

Introduces the observation-action loop that underlies many tool-using agent patterns.

OFFICIAL / DEEP REFERENCE

Building Effective Agents - Anthropic

Defines workflows versus agents and argues for simple composable patterns before complex frameworks.

Quorums and replicated reads/writes

WHAT TO LEARN - IN ORDER

- Reason about overlapping read/write sets in replicated storage.



LINEAR LEARNING PATH

1. N replicas
2. Read quorum R
3. Write quorum W
4. Consistency/availability trade-offs

DO THIS

Simulate $N=3$ with different R/W policies and node failures.

MID-SENIOR CHECKPOINT

What does $R+W>N$ guarantee, and what does it still not guarantee?

REQUIRED READING

Dynamo: Amazon Highly Available Key-value Store

Classic system covering consistent hashing, quorums, eventual consistency, vector clocks, and always-on availability.

OFFICIAL / DEEP REFERENCE

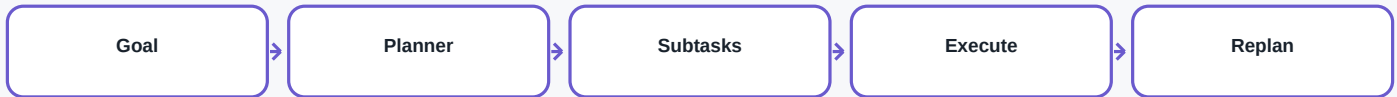
Research for Practice: Convergence - Kleppmann & Alvaro

Explores consensus versus convergence, coordination costs, and eventually consistent replicated data.

Planning and task decomposition

WHAT TO LEARN - IN ORDER

- Use planning only when decomposition improves success, parallelism, or auditability.



LINEAR LEARNING PATH

1. Plan before execution when tasks are coupled
2. Replan from evidence
3. Separate plan from actual state
4. Avoid planning trivial work

DO THIS

Give an agent a 5-part task and compare plan-first vs direct execution on success and tokens.

MID-SENIOR CHECKPOINT

When does planning add hallucinated commitments rather than useful structure?

REQUIRED READING

A Practical Guide to Building AI Agents - OpenAI

Production-oriented overview of when to use agents, how to orchestrate them, and how to apply safeguards.

OFFICIAL / DEEP REFERENCE

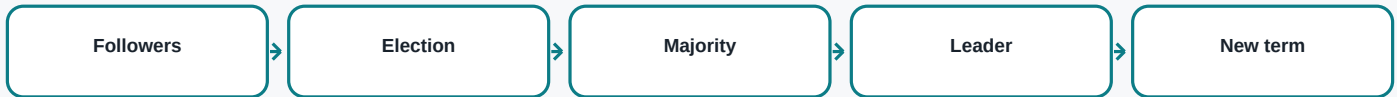
Building Effective Agents - Anthropic

Defines workflows versus agents and argues for simple composable patterns before complex frameworks.

Consensus and Raft leader election

WHAT TO LEARN - IN ORDER

- Understand why distributed replicas need an agreed authority before committing one history.



LINEAR LEARNING PATH

1. Terms
2. Follower/candidate/leader
3. Majority election
4. Split-vote retry

DO THIS

Run the Raft visualization or implement a tiny election simulator.

MID-SENIOR CHECKPOINT

Why can two nodes believe they are leaders at different times without violating safety?

REQUIRED READING

Raft: Understandable Consensus

Explains leader election, replicated logs, commitment, terms, and consensus safety.

OFFICIAL / DEEP REFERENCE

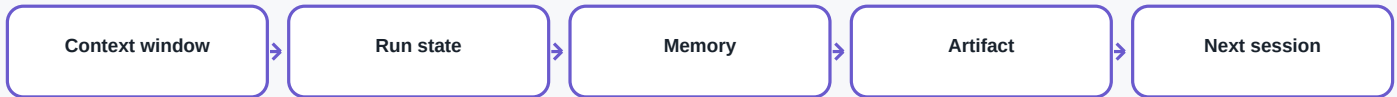
Raft Home and Visualization

Companion material, visualizations, and implementations for the Raft paper.

Context, state, memory, artifact

WHAT TO LEARN - IN ORDER

- Stop treating chat history as a universal persistence layer.



LINEAR LEARNING PATH

1. Context: what model sees now
2. State: machine-readable run position
3. Memory: reusable future guidance
4. Artifact: reviewed source of truth

DO THIS

Refactor one agent so durable task state is stored separately from conversation messages.

MID-SENIOR CHECKPOINT

Which facts must survive even if every model message is deleted?

REQUIRED READING

[Effective Context Engineering for AI Agents - Anthropic](#)

Explains compaction, structured note-taking, and context curation for long-horizon agents.

OFFICIAL / DEEP REFERENCE

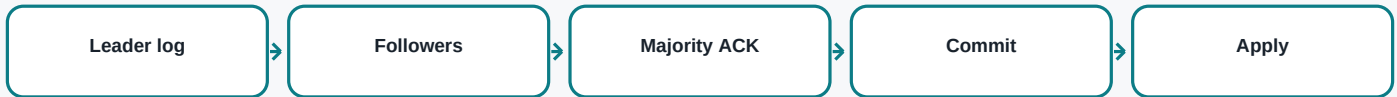
[Building Reliable Agents with Memory and Compaction - OpenAI](#)

Separates active-run compaction, reusable memory, and the reviewed artifact as distinct responsibilities.

Raft log replication and commit

WHAT TO LEARN - IN ORDER

- Follow one write from leader append through follower replication to majority commitment.



LINEAR LEARNING PATH

1. Log index/term
2. AppendEntries
3. Majority acknowledgement
4. Commit index and application

DO THIS

Simulate one leader failure before and after majority replication.

MID-SENIOR CHECKPOINT

Why is "written on the leader" different from "committed"?

REQUIRED READING

Raft: Understandable Consensus

Explains leader election, replicated logs, commitment, terms, and consensus safety.

OFFICIAL / DEEP REFERENCE

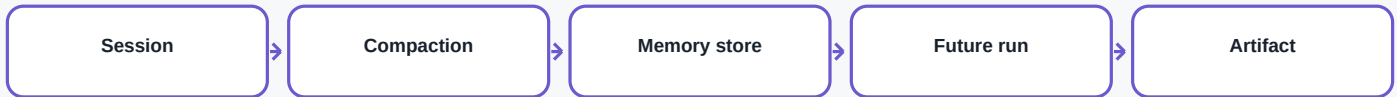
Raft Home

Visual and implementation resources that make election/log behavior easier to inspect.

Agent memory architecture

WHAT TO LEARN - IN ORDER

- Design memory around reusable behavior and preferences, not indiscriminate storage of every conversation.



LINEAR LEARNING PATH

1. Short-term session context
2. Compaction
3. Long-term memory
4. Provenance and deletion boundaries

DO THIS

Add explicit memory write/read rules to an agent and test whether irrelevant memories pollute later runs.

MID-SENIOR CHECKPOINT

What deserves long-term memory, and what should remain only in the reviewed artifact?

REQUIRED READING

Building Reliable Agents with Memory and Compaction - OpenAI

Separates active-run compaction, reusable memory, and the reviewed artifact as distinct responsibilities.

OFFICIAL / DEEP REFERENCE

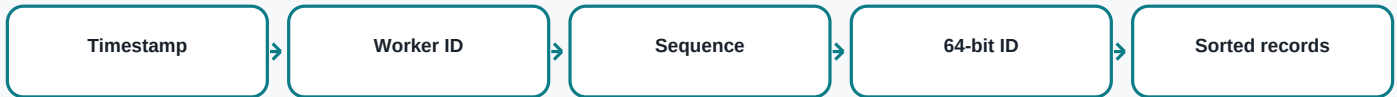
Effective Context Engineering for AI Agents - Anthropic

Explains compaction, structured note-taking, and context curation for long-horizon agents.

Distributed unique IDs

WHAT TO LEARN - IN ORDER

- Generate IDs without a single database sequence becoming a global bottleneck.



LINEAR LEARNING PATH

1. Timestamp bits
2. Worker/machine identifier
3. Per-time-unit sequence
4. Clock rollback risk

DO THIS

Implement a Snowflake-style 64-bit ID generator and decode generated IDs.

MID-SENIOR CHECKPOINT

What happens if two workers accidentally share the same worker ID?

REQUIRED READING

Twitter Snowflake

Reference design for generating sortable globally unique IDs without a single database sequence.

OFFICIAL / DEEP REFERENCE

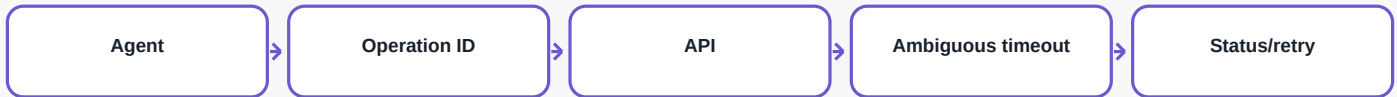
Time, Clocks, and the Ordering of Events - Lamport

Foundational paper for reasoning about event ordering and causality without relying on synchronized wall clocks.

Reliable tool execution and idempotency

WHAT TO LEARN - IN ORDER

- A timeout is ambiguous: the side effect may have happened even if the agent never received the response.



LINEAR LEARNING PATH

1. Operation IDs
2. Idempotency keys
3. Status lookup
4. Retry only when semantics are known

DO THIS

Wrap one side-effecting tool with `operation_id` and make repeated retries produce one business effect.

MID-SENIOR CHECKPOINT

How should an agent recover when an API times out after committing the operation?

REQUIRED READING

Writing Effective Tools for Agents - Anthropic

Shows how tool naming, boundaries, output design, and evals affect agent performance.

OFFICIAL / DEEP REFERENCE

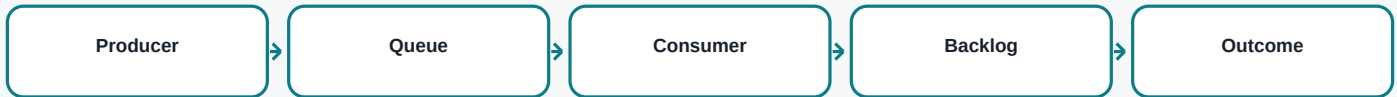
Timeouts, Retries and Backoff with Jitter - AWS Builders Library

Canonical reliability guidance for safe retry policies, bounded attempts, exponential backoff, and jitter.

Queues and asynchronous processing

WHAT TO LEARN - IN ORDER

- Use queues to decouple arrival time from processing time while remembering that buffering is not capacity.



LINEAR LEARNING PATH

1. Producer/consumer decoupling
2. Queue depth
3. Service rate vs arrival rate
4. Visibility/acknowledgement

DO THIS

Build a worker queue and deliberately make producers 2x faster than consumers.

MID-SENIOR CHECKPOINT

When does a queue make an outage worse by preserving too much stale work?

REQUIRED READING

Avoiding Insurmountable Queue Backlogs - AWS Builders Library

Explains backlog growth, capacity mismatch, stale work, and why queues do not create processing capacity.

OFFICIAL / DEEP REFERENCE

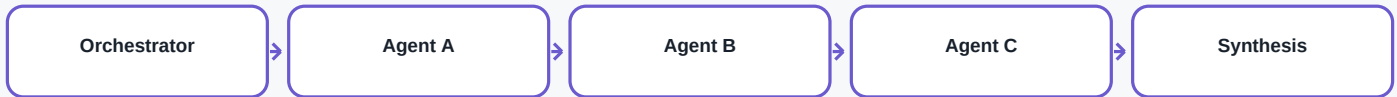
Timeouts, Retries and Backoff with Jitter - AWS Builders Library

Canonical reliability guidance for retries, timeouts, exponential backoff, jitter, and amplification control.

Single-agent vs multi-agent systems

WHAT TO LEARN - IN ORDER

- Parallel agents help when work can truly be decomposed; otherwise they add coordination cost and inconsistency.



LINEAR LEARNING PATH

1. Parallel independent subtasks
2. Specialized context
3. Coordinator/synthesizer
4. Shared-state hazards

DO THIS

Run one research task with one agent and with 3 parallel subagents; compare cost, latency, and answer quality.

MID-SENIOR CHECKPOINT

Which dependencies make parallel agents unsafe or wasteful?

REQUIRED READING

How We Built Our Multi-Agent Research System - Anthropic

Production lessons on parallel agents, orchestration, coordination, evaluation, and reliability.

OFFICIAL / DEEP REFERENCE

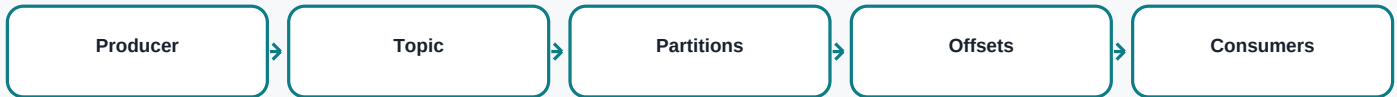
Building Effective Agents - Anthropic

Defines workflows versus agents and argues for simple composable patterns before complex frameworks.

Kafka topics, partitions, and offsets

WHAT TO LEARN - IN ORDER

- Understand Kafka as a replicated ordered log, not a magical message bus.



LINEAR LEARNING PATH

1. Partition is the ordering unit
2. Offsets identify positions
3. Producer partitioning
4. Replication and durability

DO THIS

Produce keyed events to a 3-partition topic and observe ordering and offsets.

MID-SENIOR CHECKPOINT

Why can Kafka preserve per-user ordering but not global ordering at high scale?

REQUIRED READING

Apache Kafka Design

Official explanation of partitions, persistence, batching, consumer offsets, replication, and delivery guarantees.

OFFICIAL / DEEP REFERENCE

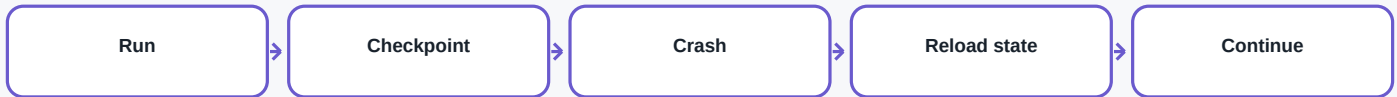
Apache Kafka Documentation

Operational reference for consumer groups, producers, brokers, configuration, and ecosystem behavior.

Durable and resumable agent execution

WHAT TO LEARN - IN ORDER

- Long-running work must survive context limits, process crashes, and human pauses.



LINEAR LEARNING PATH

1. Checkpoint explicit progress
2. Persist artifacts and task ledger
3. Resume from durable state
4. Verify before declaring completion

DO THIS

Kill an agent process halfway through a 10-step task and resume from stored state without repeating side effects.

MID-SENIOR CHECKPOINT

What is the minimal state required to resume safely?

REQUIRED READING

Effective Harnesses for Long-running Agents - Anthropic

Shows how agents preserve progress across context windows using explicit artifacts and incremental work.

OFFICIAL / DEEP REFERENCE

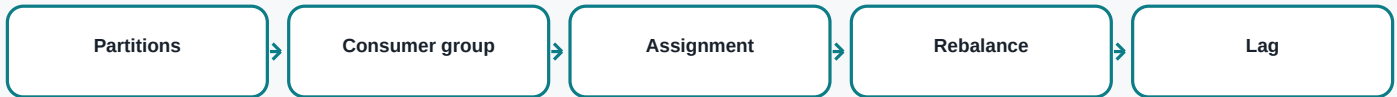
Background Mode - OpenAI

Shows how to model long-running AI work as asynchronous jobs rather than fragile client connections.

Kafka consumer groups and rebalancing

WHAT TO LEARN - IN ORDER

- Understand how partitions are assigned to workers and what happens when fleet membership changes.



LINEAR LEARNING PATH

1. One partition per active consumer within a group
2. Rebalance on membership change
3. Offset commits
4. Slow consumer effects

DO THIS

Start/stop consumers and observe partition reassignment and lag.

MID-SENIOR CHECKPOINT

Why does adding consumers beyond partition count stop increasing parallelism?

REQUIRED READING

Apache Kafka Documentation

Operational reference for consumer groups, producers, brokers, configuration, and ecosystem behavior.

OFFICIAL / DEEP REFERENCE

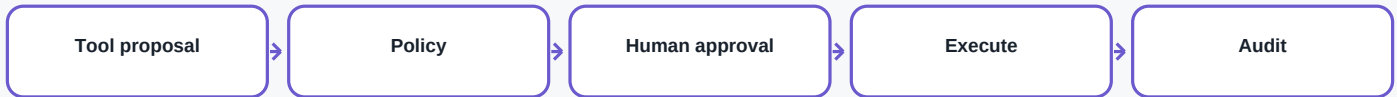
Apache Kafka Design

Official explanation of partitions, persistence, batching, consumer offsets, replication, and delivery guarantees.

Guardrails, permissions, and human approval

WHAT TO LEARN - IN ORDER

- Separate automated policy checks from explicit approval for high-risk or irreversible actions.



LINEAR LEARNING PATH

1. Risk classification
2. Automatic guardrail
3. Human gate
4. Resume same durable run

DO THIS

Assign risk/reversibility metadata to 10 tools and implement a simple approval policy engine.

MID-SENIOR CHECKPOINT

Which actions should remain impossible even after a user clicks Approve?

REQUIRED READING

Guardrails and Human Approvals - OpenAI

Covers automated safety checks and explicit human approval for sensitive tool actions.

OFFICIAL / DEEP REFERENCE

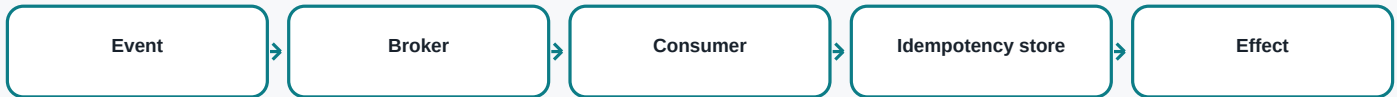
How We Contain Claude Across Products - Anthropic

Production lessons on sandboxing, scoped credentials, network boundaries, and limiting agent blast radius.

Delivery semantics and idempotent consumers

WHAT TO LEARN - IN ORDER

- Retries make duplicates normal; business effects must tolerate redelivery.



LINEAR LEARNING PATH

1. At-most-once
2. At-least-once
3. Duplicate delivery
4. Idempotent consumer state

DO THIS

Process the same event twice and design a deduplication or idempotency key strategy.

MID-SENIOR CHECKPOINT

What exactly does "exactly once" mean at the boundary of an external side effect?

REQUIRED READING

Apache Kafka Design

Official explanation of partitions, persistence, batching, consumer offsets, replication, and delivery guarantees.

OFFICIAL / DEEP REFERENCE

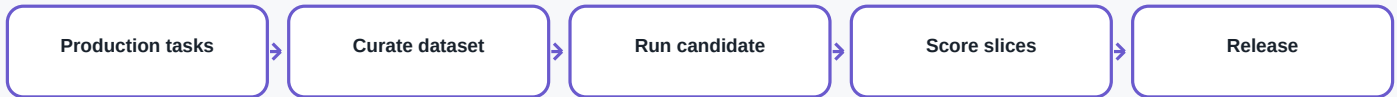
Idempotent Requests - Stripe

A concrete API-side example of turning duplicate retries into one logical effect.

Designing an AI eval dataset

WHAT TO LEARN - IN ORDER

- An eval set is a product specification expressed as examples and measurements.



LINEAR LEARNING PATH

1. Core/common cases
2. Historical failures
3. Edge/adversarial cases
4. Slice results instead of one average

DO THIS

Create 50 eval cases for one feature and tag each by failure category and importance.

MID-SENIOR CHECKPOINT

What important user behavior is absent from your test set?

REQUIRED READING

Demystifying Evals for AI Agents - Anthropic

Explains why multi-turn tool-using agents require richer evals than single-response models.

OFFICIAL / DEEP REFERENCE

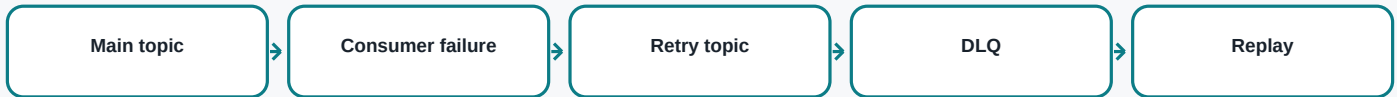
AI Product Engineering Notes - Hamel Husain

Practical guidance on error analysis, eval design, human review, and building AI products around real failures.

Retry topology and dead-letter queues

WHAT TO LEARN - IN ORDER

- Prevent poison messages from blocking healthy real-time traffic.



LINEAR LEARNING PATH

1. Transient vs permanent errors
2. Delayed retry topics
3. Max attempt policy
4. DLQ triage/replay

DO THIS

Create main, retry, and DLQ topics and force one poison message through the full path.

MID-SENIOR CHECKPOINT

What information must a DLQ record preserve for safe reprocessing?

REQUIRED READING

Reliable Reprocessing and Dead Letter Queues with Kafka - Uber

Shows how retry topics and DLQs prevent poison messages from blocking real-time consumption.

OFFICIAL / DEEP REFERENCE

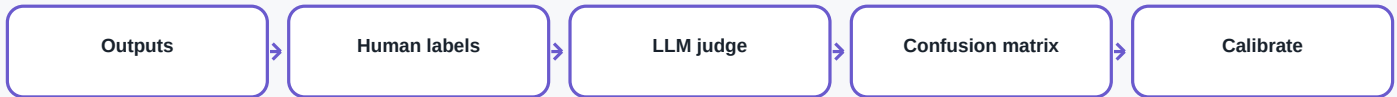
Apache Kafka Design

Official explanation of partitions, persistence, batching, consumer offsets, replication, and delivery guarantees.

LLM-as-judge calibration

WHAT TO LEARN - IN ORDER

- Automated judges are useful only after proving they approximate the human criterion you care about.



LINEAR LEARNING PATH

1. Human-labeled calibration set
2. Precision/recall
3. Judge bias
4. Disagreement review

DO THIS

Label 50 outputs manually, run an LLM judge, and calculate a confusion matrix.

MID-SENIOR CHECKPOINT

Can a judge with 95% accuracy still be unsafe for your release gate?

REQUIRED READING

Putting Evals Into Production - Hamel Husain

Shows how human labels, evaluator validation, and production samples become a continuous quality loop.

OFFICIAL / DEEP REFERENCE

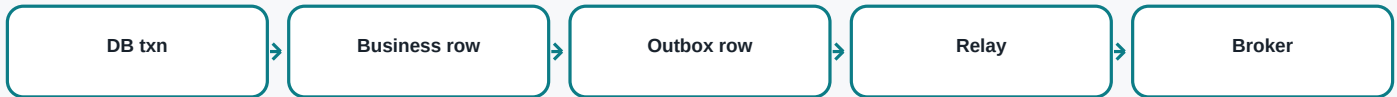
Demystifying Evals for AI Agents - Anthropic

Explains why multi-turn tool-using agents require richer evals than single-response models.

Transactional outbox

WHAT TO LEARN - IN ORDER

- Avoid unsafe dual writes when one business operation must update a DB and publish an event.



LINEAR LEARNING PATH

1. Business row + outbox row in one transaction
2. Relay/CDC publishes later
3. Idempotent publication
4. Consumer deduplication

DO THIS

Implement orders + outbox table and a relay that publishes unsent rows.

MID-SENIOR CHECKPOINT

What happens if the relay publishes successfully and crashes before marking the row sent?

REQUIRED READING

Transactional Outbox Pattern - microservices.io

Explains how to atomically persist business changes and outgoing events without unsafe dual writes.

OFFICIAL / DEEP REFERENCE

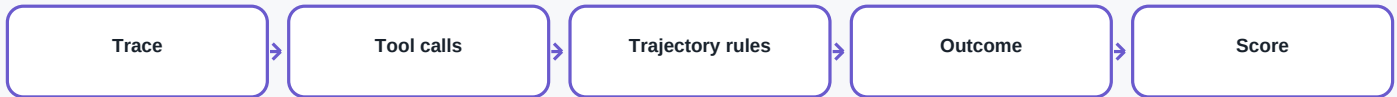
Debezium Documentation

Official CDC reference for streaming database changes into event systems.

Trajectory and tool-call evals

WHAT TO LEARN - IN ORDER

- For agents, the path matters: an apparently correct answer may hide unsafe or wasteful actions.



LINEAR LEARNING PATH

1. Outcome metrics
2. Tool-selection correctness
3. Unnecessary steps
4. Handoff/trajectory checks

DO THIS

Add assertions for forbidden tools, maximum steps, and required evidence to an agent eval.

MID-SENIOR CHECKPOINT

When should a correct final answer still fail the eval?

REQUIRED READING

Agent Evals - OpenAI

Reference for evaluating agent outcomes, trajectories, tools, and workflow behavior.

OFFICIAL / DEEP REFERENCE

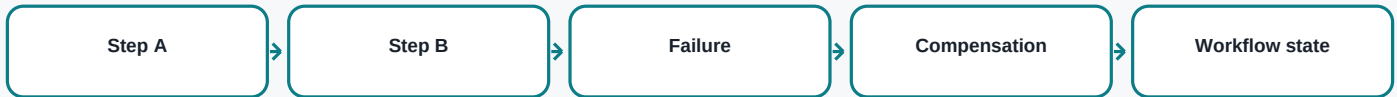
Agent Tracing - OpenAI

Shows how to inspect agent runs as traces composed of model, tool, and orchestration spans.

Saga pattern and compensating actions

WHAT TO LEARN - IN ORDER

- Coordinate multi-service workflows without pretending one global ACID transaction exists.



LINEAR LEARNING PATH

1. Local transactions
2. Workflow state
3. Compensation
4. Irreversible step handling

DO THIS

Model order->payment->inventory->shipping with compensations for failures at each stage.

MID-SENIOR CHECKPOINT

Which actions cannot truly be "rolled back" and how does the design account for them?

REQUIRED READING

[Saga Distributed Transactions - Azure Architecture Center](#)

Shows how long-running multi-service workflows use local transactions and compensating actions.

OFFICIAL / DEEP REFERENCE

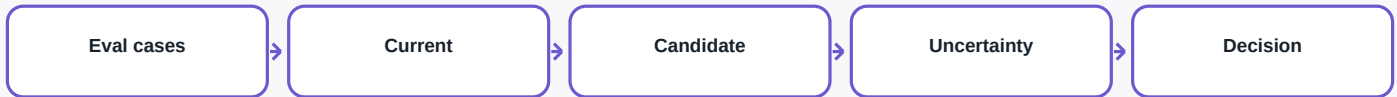
[Transactional Outbox Pattern - microservices.io](#)

Explains how to atomically persist business changes and outgoing events without unsafe dual writes.

Statistical evals and confidence

WHAT TO LEARN - IN ORDER

- Small score changes are meaningless without understanding sample size, variance, and paired comparisons.



LINEAR LEARNING PATH

1. Paired evals
2. Confidence intervals intuition
3. Failure severity
4. Avoid repeated test-set tuning

DO THIS

Bootstrap the difference between two candidates on a fixed eval set.

MID-SENIOR CHECKPOINT

Would you ship a +1% average gain if the candidate regresses a critical slice?

REQUIRED READING

Demystifying Evals for AI Agents - Anthropic

Explains why multi-turn tool-using agents require richer evals than single-response models.

OFFICIAL / DEEP REFERENCE

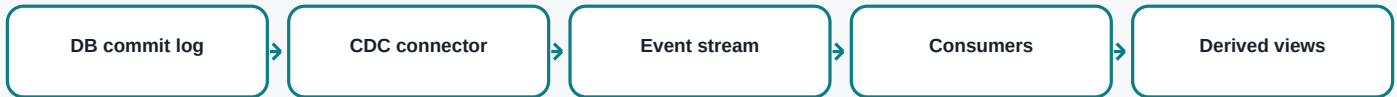
AI Product Engineering Notes - Hamel Husain

Practical guidance on error analysis, eval design, human review, and building AI products around real failures.

Change Data Capture

WHAT TO LEARN - IN ORDER

- Turn committed database changes into an event stream without forcing every service to dual-write events manually.



LINEAR LEARNING PATH

1. Read transaction log
2. Emit ordered change events
3. Schema evolution
4. Replay and downstream lag

DO THIS

Run Debezium against a local DB and observe inserts/updates as events.

MID-SENIOR CHECKPOINT

What consistency assumptions can consumers make about CDC ordering across tables?

REQUIRED READING

Debezium Documentation

Official CDC reference for streaming database changes into event systems.

OFFICIAL / DEEP REFERENCE

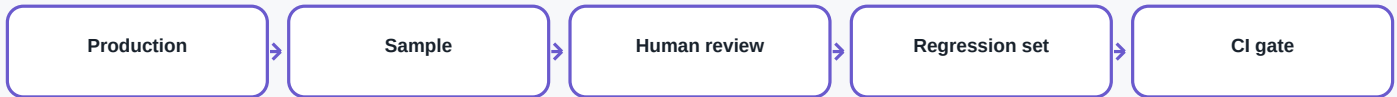
Transactional Outbox Pattern - microservices.io

Explains how to atomically persist business changes and outgoing events without unsafe dual writes.

Continuous production eval flywheel

WHAT TO LEARN - IN ORDER

- Production failures should become reusable regression cases instead of one-off prompt tweaks.



LINEAR LEARNING PATH

1. Sample live traces
2. Human review
3. Convert failures into eval cases
4. Gate future changes

DO THIS

Pick 10 real/representative traces, label failure causes, and add them to CI evals.

MID-SENIOR CHECKPOINT

How do you prevent the eval set from becoming a pile of obsolete edge cases?

REQUIRED READING

Putting Evals Into Production - Hamel Husain

Shows how human labels, evaluator validation, and production samples become a continuous quality loop.

OFFICIAL / DEEP REFERENCE

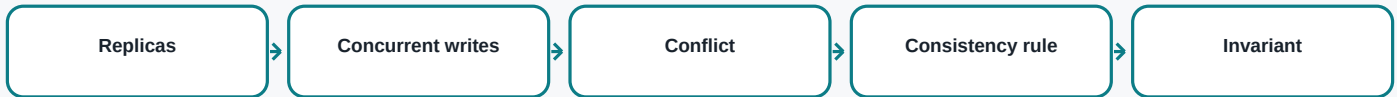
Build an Agent Improvement Loop - OpenAI

Connects real traces, human/model feedback, reusable evals, and reviewed harness changes into an improvement flywheel.

Strong vs eventual consistency

WHAT TO LEARN - IN ORDER

- Choose semantics per invariant rather than labeling an entire product "strong" or "eventual".



LINEAR LEARNING PATH

1. Stale reads
2. Concurrent writes
3. Authoritative owner
4. Invariant-driven coordination

DO THIS

Classify presence, payments, likes, inventory, and collaborative edits by consistency needs.

MID-SENIOR CHECKPOINT

Which business invariant forces coordination during a network partition?

REQUIRED READING

Research for Practice: Convergence - Kleppmann & Alvaro

Explores consensus versus convergence, coordination costs, and eventually consistent replicated data.

OFFICIAL / DEEP REFERENCE

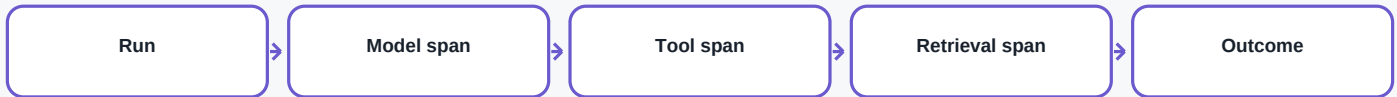
Dynamo: Amazon Highly Available Key-value Store

Classic system covering consistent hashing, quorums, eventual consistency, vector clocks, and always-on availability.

AI tracing and run observability

WHAT TO LEARN - IN ORDER

- Trace the complete run so failures can be attributed to model calls, retrieval, tools, routing, or external dependencies.



LINEAR LEARNING PATH

1. Trace/run/span hierarchy
2. Record model/tool timings
3. Capture errors and retries
4. Keep sensitive data policies explicit

DO THIS

Instrument one agent run and create a trace waterfall with model and tool spans.

MID-SENIOR CHECKPOINT

What production question can your logs answer that your traces cannot?

REQUIRED READING

Agent Tracing - OpenAI

Shows how to inspect agent runs as traces composed of model, tool, and orchestration spans.

OFFICIAL / DEEP REFERENCE

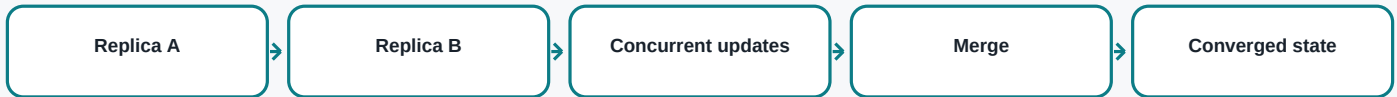
OpenTelemetry Documentation

Vendor-neutral reference for traces, metrics, logs, context propagation, and telemetry pipelines.

Convergence and CRDT intuition

WHAT TO LEARN - IN ORDER

- Learn how some replicated data types can accept concurrent updates and deterministically merge later.



LINEAR LEARNING PATH

1. Convergence vs coordination
2. Merge semantics
3. Commutative/idempotent updates
4. Invariant limitations

DO THIS

Implement a G-Counter on two replicas and merge updates in different orders.

MID-SENIOR CHECKPOINT

Why does replica convergence not imply that an inventory invariant was preserved?

REQUIRED READING

Research for Practice: Convergence - Kleppmann & Alvaro

Explores consensus versus convergence, coordination costs, and eventually consistent replicated data.

OFFICIAL / DEEP REFERENCE

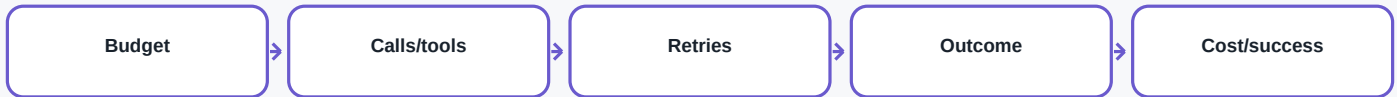
Dynamo: Amazon Highly Available Key-value Store

Classic system covering consistent hashing, quorums, eventual consistency, vector clocks, and always-on availability.

Cost per successful task

WHAT TO LEARN - IN ORDER

- Optimize the cost of achieving a correct user outcome, not just cost per API call.



LINEAR LEARNING PATH

1. Token/request accounting
2. Model/tool call count
3. Failure/retry cost
4. Cost per successful task

DO THIS

Add a per-run budget counter and stop/degrade when expected next-step cost exceeds remaining budget.

MID-SENIOR CHECKPOINT

When can a more expensive model reduce total cost by avoiding retries or tool mistakes?

REQUIRED READING

Cost Optimization - OpenAI

Explains reducing requests, tokens, model size, and using batch/flex processing for lower cost.

OFFICIAL / DEEP REFERENCE

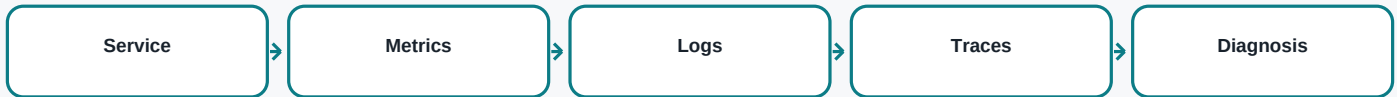
API Deployment Checklist - OpenAI

Current deployment checklist spanning model choice, reasoning budgets, tools, compaction, caching, and reliability.

Metrics, logs, and distributed traces

WHAT TO LEARN - IN ORDER

- Know which telemetry answers aggregate health, event detail, and request-path questions.



LINEAR LEARNING PATH

1. Metrics for trends/alerts
2. Logs for event detail
3. Traces for causal request paths
4. Correlation IDs/context propagation

DO THIS

Instrument one endpoint with all three signal types and debug one synthetic failure.

MID-SENIOR CHECKPOINT

Which signal would you query first for a p99 latency regression?

REQUIRED READING

Monitoring Distributed Systems - Google SRE

Introduces latency, traffic, errors, saturation, and symptom-oriented monitoring.

OFFICIAL / DEEP REFERENCE

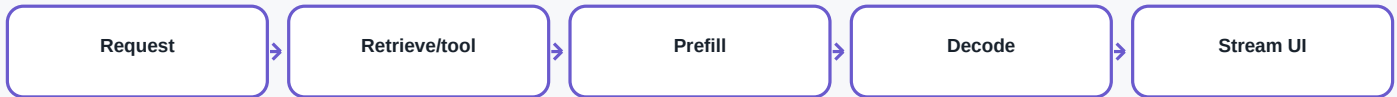
OpenTelemetry Traces

Official trace model: spans, trace context, parent-child relationships, and propagation.

Latency breakdown: TTFT, token speed, tools

WHAT TO LEARN - IN ORDER

- Treat end-to-end latency as a critical path rather than blaming the model for every slow response.



LINEAR LEARNING PATH

1. Network/queue
2. Prefill/TTFT
3. Decode/TPOT
4. Sequential vs parallel tools

DO THIS

Instrument a request and produce a millisecond waterfall for every major stage.

MID-SENIOR CHECKPOINT

Which optimization changes user-perceived latency without changing total compute time?

REQUIRED READING

Latency Optimization - OpenAI

Explains how model choice, token count, request count, parallelism, and UI design affect end-to-end latency.

OFFICIAL / DEEP REFERENCE

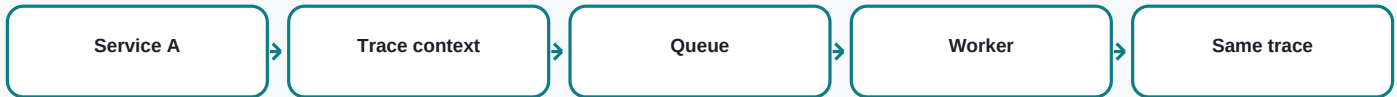
Production Best Practices - OpenAI

Production checklist for reliability, latency, usage limits, and operational safeguards.

OpenTelemetry and context propagation

WHAT TO LEARN - IN ORDER

- Standardize trace/span context across services so one request stays correlated through process boundaries.



LINEAR LEARNING PATH

1. Trace ID
2. Span ID
3. Parent-child relations
4. Propagation through HTTP/queues

DO THIS

Propagate a trace context from HTTP request to background worker.

MID-SENIOR CHECKPOINT

What breaks in incident debugging when async jobs create unrelated traces?

REQUIRED READING

OpenTelemetry Traces

Official trace model: spans, trace context, parent-child relationships, and propagation.

OFFICIAL / DEEP REFERENCE

Dapper: Large-Scale Distributed Tracing Infrastructure - Google

Foundational tracing paper for reconstructing one request across many distributed services.

AI SLOs and service-quality metrics

WHAT TO LEARN - IN ORDER

- Define reliability around user-visible AI outcomes rather than only provider uptime.



LINEAR LEARNING PATH

1. Availability of usable answers
2. Latency SLO
3. Quality/task-success metric
4. Cost/abuse operational limits

DO THIS

Write an SLO for one AI endpoint including success, latency, and degraded-mode behavior.

MID-SENIOR CHECKPOINT

Can an API be 99.99% available while the AI product is effectively broken?

REQUIRED READING

Production Best Practices - OpenAI

Production checklist for reliability, latency, usage limits, and operational safeguards.

OFFICIAL / DEEP REFERENCE

API Deployment Checklist - OpenAI

Current deployment checklist spanning model choice, reasoning budgets, tools, compaction, caching, and reliability.

SLIs, SLOs, and error budgets

WHAT TO LEARN - IN ORDER

- Turn reliability into explicit objectives and a budget for failure/change.



LINEAR LEARNING PATH

1. Choose user-journey SLI
2. Set target SLO
3. Calculate error budget
4. Use budget to govern release pace

DO THIS

Compute monthly downtime/error budget for 99.9% and 99.99% services.

MID-SENIOR CHECKPOINT

Which SLI most closely matches what users actually experience?

REQUIRED READING

Alerting on SLOs - Google SRE Workbook

Shows how error budgets and multi-window burn-rate alerts create actionable reliability paging.

OFFICIAL / DEEP REFERENCE

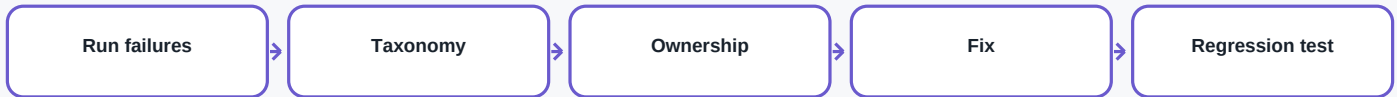
Monitoring Distributed Systems - Google SRE

Introduces latency, traffic, errors, saturation, and symptom-oriented monitoring.

AI reliability budgets and failure taxonomy

WHAT TO LEARN - IN ORDER

- Classify failures so the team knows whether to change prompt, retrieval, model, tool, infra, or product semantics.



LINEAR LEARNING PATH

1. Model failure
2. Retrieval failure
3. Tool/dependency failure
4. Policy/product ambiguity

DO THIS

Label 50 failures by root-cause family and count where engineering effort should go.

MID-SENIOR CHECKPOINT

Which failure classes are detectable automatically versus only through expert review?

REQUIRED READING

Common Pitfalls When Building Generative AI Applications - Chip Huyen

Production lessons on premature complexity, evaluation, human review, and long-tail failures.

OFFICIAL / DEEP REFERENCE

AI Product Engineering Notes - Hamel Husain

Practical guidance on error analysis, eval design, human review, and building AI products around real failures.

Burn-rate alerting

WHAT TO LEARN - IN ORDER

- Page on how quickly reliability budget is being consumed, not arbitrary one-off thresholds.



LINEAR LEARNING PATH

1. Error-budget burn rate
2. Fast vs slow windows
3. Multi-window alerting
4. Avoid noisy paging

DO THIS

For a 99.9% SLO, calculate burn rate under 1%, 5%, and 20% error rates.

MID-SENIOR CHECKPOINT

Why is a short 10x burn spike different from a 2x burn sustained for hours?

REQUIRED READING

Alerting on SLOs - Google SRE Workbook

Shows how error budgets and multi-window burn-rate alerts create actionable reliability paging.

OFFICIAL / DEEP REFERENCE

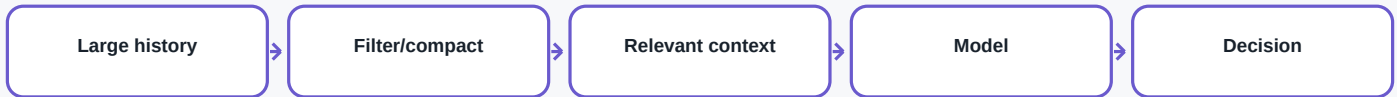
Monitoring Distributed Systems - Google SRE

Introduces latency, traffic, errors, saturation, and symptom-oriented monitoring.

Context engineering and context budgets

WHAT TO LEARN - IN ORDER

- Select the minimum useful evidence for the next decision rather than filling the context window because space exists.



LINEAR LEARNING PATH

1. Context relevance
2. Tool-output pruning
3. Compaction
4. External artifacts instead of transcript replay

DO THIS

Measure one agent task with full history vs curated context and compare tokens and success.

MID-SENIOR CHECKPOINT

What information is useful to store but harmful to keep in active context?

REQUIRED READING

[Effective Context Engineering for AI Agents - Anthropic](#)

Explains compaction, structured note-taking, and context curation for long-horizon agents.

OFFICIAL / DEEP REFERENCE

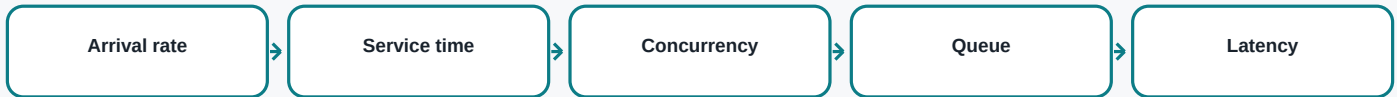
[Building Reliable Agents with Memory and Compaction - OpenAI](#)

Separates active-run compaction, reusable memory, and the reviewed artifact as distinct responsibilities.

Little's Law and queueing intuition

WHAT TO LEARN - IN ORDER

- Connect arrival rate, work duration, and concurrency so capacity reasoning becomes quantitative.



LINEAR LEARNING PATH

1. $L = \lambda W$
2. Concurrency grows with service time
3. Saturation creates queues
4. Latency feedback loops

DO THIS

Calculate concurrency for 100 req/s at 50ms, 500ms, and 5s service times.

MID-SENIOR CHECKPOINT

Why can latency increase cause overload even when incoming traffic is unchanged?

REQUIRED READING

Avoiding Insurmountable Queue Backlogs - AWS Builders Library

Explains backlog growth, capacity mismatch, stale work, and why queues do not create processing capacity.

OFFICIAL / DEEP REFERENCE

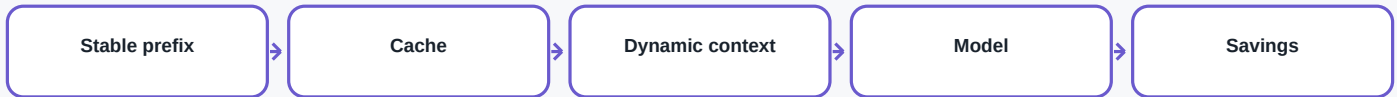
Scaling Your API with Rate Limiters - Stripe

Production design for request rate limits, concurrency limits, fleet protection, and worker-utilization controls.

Prompt and context caching

WHAT TO LEARN - IN ORDER

- Exploit repeated stable prefixes, tool schemas, and shared instructions without confusing caching with memory.



LINEAR LEARNING PATH

1. Stable prefix
2. Cache hit/miss
3. Dynamic suffix
4. Cost/latency measurement

DO THIS

Reorder one prompt so stable content comes first and compare cache hit behavior.

MID-SENIOR CHECKPOINT

Which personalization fields accidentally destroy cache reuse?

REQUIRED READING

Prompt Caching - OpenAI

Explains stable-prefix reuse for lower input cost and lower prefill latency.

OFFICIAL / DEEP REFERENCE

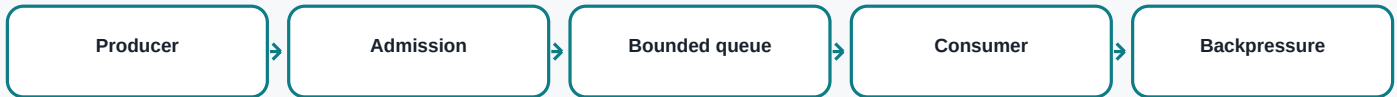
API Deployment Checklist - OpenAI

Current deployment checklist spanning model choice, reasoning budgets, tools, compaction, caching, and reliability.

Backpressure and bounded queues

WHAT TO LEARN - IN ORDER

- Prevent downstream overload by slowing/rejecting upstream work instead of accepting infinite backlog.



LINEAR LEARNING PATH

1. Bound queue size
2. Reject or shed
3. Producer throttling
4. Age-of-work limits

DO THIS

Add a bounded queue to a worker system and compare behavior with an unbounded queue during overload.

MID-SENIOR CHECKPOINT

When should the producer fail fast instead of enqueueing work?

REQUIRED READING

Avoiding Insurmountable Queue Backlogs - AWS Builders Library

Explains backlog growth, capacity mismatch, stale work, and why queues do not create processing capacity.

OFFICIAL / DEEP REFERENCE

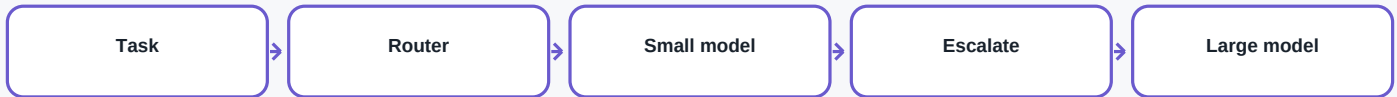
Addressing Cascading Failures - Google SRE

Explains overload, retry storms, load shedding, capacity planning, and preventing failure propagation.

Model routing and cascades

WHAT TO LEARN - IN ORDER

- Use workload-specific routing to balance quality, latency, and cost.



LINEAR LEARNING PATH

1. Rule-based routing
2. Difficulty/risk classifier
3. Small->large escalation
4. Evaluate routing policy itself

DO THIS

Create a 2-model cascade and measure fallback rate plus quality/cost.

MID-SENIOR CHECKPOINT

What happens when the router is confidently wrong on the hardest tasks?

REQUIRED READING

API Deployment Checklist - OpenAI

Current deployment checklist spanning model choice, reasoning budgets, tools, compaction, caching, and reliability.

OFFICIAL / DEEP REFERENCE

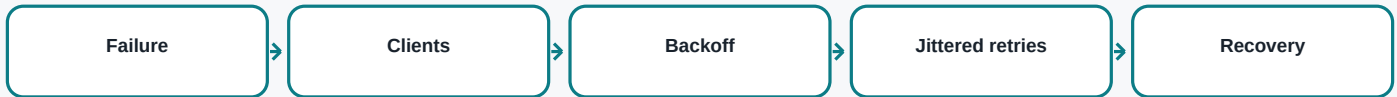
Model Selection Guide - OpenAI

Useful for thinking about workload-model fit rather than defaulting to the largest model.

Retries, exponential backoff, and jitter

WHAT TO LEARN - IN ORDER

- Retries are load multipliers; safe systems bound them and decorrelate clients.



LINEAR LEARNING PATH

1. Transient vs permanent errors
2. Retry budget
3. Exponential backoff
4. Jitter

DO THIS

Simulate 100 clients retrying together, then add jitter and compare request spikes.

MID-SENIOR CHECKPOINT

Where in a multi-layer stack should retries live to avoid multiplication?

REQUIRED READING

Timeouts, Retries and Backoff with Jitter - AWS Builders Library

Canonical reliability guidance for retries, timeouts, exponential backoff, jitter, and amplification control.

OFFICIAL / DEEP REFERENCE

Addressing Cascading Failures - Google SRE

Explains overload, retry storms, load shedding, capacity planning, and preventing failure propagation.

Provider failover and graceful degradation

WHAT TO LEARN - IN ORDER

- Define what the product does when the preferred model/provider is slow, unavailable, or over budget.



LINEAR LEARNING PATH

1. Deadline-aware execution
2. Fallback model/provider
3. Reduced feature mode
4. Never violate the same safety/eval contract

DO THIS

Create a degradation ladder: full agent -> reduced tools -> smaller model -> partial result.

MID-SENIOR CHECKPOINT

When is returning a partial verified result better than switching to a weaker model?

REQUIRED READING

Production Best Practices - OpenAI

Production checklist for reliability, latency, usage limits, and operational safeguards.

OFFICIAL / DEEP REFERENCE

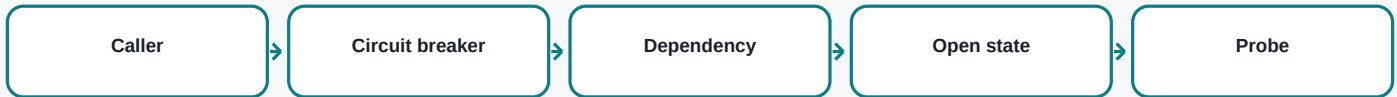
Latency Optimization - OpenAI

Explains how model choice, token count, request count, parallelism, and UI design affect end-to-end latency.

Circuit breakers and bulkheads

WHAT TO LEARN - IN ORDER

- Stop repeated calls to a failing dependency and isolate resources so one dependency cannot consume the whole fleet.



LINEAR LEARNING PATH

1. Closed/open/half-open
2. Failure threshold
3. Probe recovery
4. Pool/resource isolation

DO THIS

Wrap one unstable HTTP dependency with a circuit breaker and separate concurrency pool.

MID-SENIOR CHECKPOINT

How can a circuit breaker make recovery slower if thresholds are poorly chosen?

REQUIRED READING

Circuit Breaker Pattern - Microsoft

Explains closed/open/half-open states and how clients stop hammering failing dependencies.

OFFICIAL / DEEP REFERENCE

Addressing Cascading Failures - Google SRE

Explains overload, retry storms, load shedding, capacity planning, and preventing failure propagation.

Long-running and background AI jobs

WHAT TO LEARN - IN ORDER

- Model lengthy reasoning as a durable job with status, cancellation, and resumption.



LINEAR LEARNING PATH

1. Job ID/state machine
2. Client connection independent of job lifetime
3. Polling/subscription
4. Cancellation and expiry

DO THIS

Build POST /runs returning a job_id and a GET /runs/{id} status endpoint.

MID-SENIOR CHECKPOINT

What should happen if the user disconnects after the job has already started?

REQUIRED READING

Background Mode - OpenAI

Shows how to model long-running AI work as asynchronous jobs rather than fragile client connections.

OFFICIAL / DEEP REFERENCE

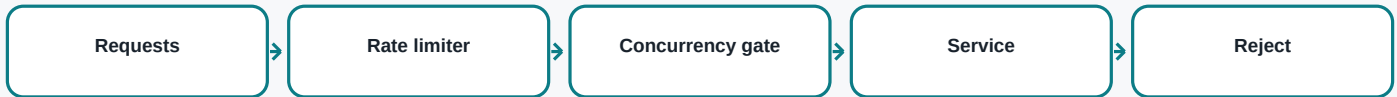
Effective Harnesses for Long-running Agents - Anthropic

Shows how agents preserve progress across context windows using explicit artifacts and incremental work.

Rate limits and admission control

WHAT TO LEARN - IN ORDER

- Protect finite resources before saturation creates queues, timeouts, and retry storms.



LINEAR LEARNING PATH

1. Request rate vs concurrency
2. Per-user fairness
3. Global capacity protection
4. 429/retry guidance

DO THIS

Add both token-bucket rate limiting and concurrency limiting to one API.

MID-SENIOR CHECKPOINT

Why can request-rate limits fail to protect a service whose requests have highly variable cost?

REQUIRED READING

Scaling Your API with Rate Limiters - Stripe

Production design for request rate limits, concurrency limits, fleet protection, and worker-utilization controls.

OFFICIAL / DEEP REFERENCE

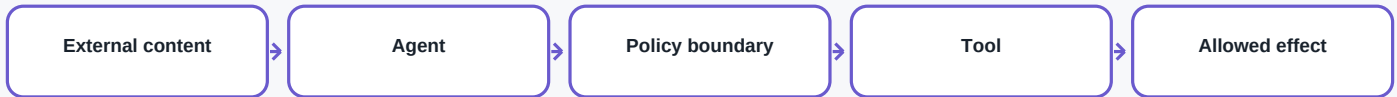
NGINX Request Limiting

Implementation-level reference for request-rate control at a proxy layer.

Prompt injection and untrusted context

WHAT TO LEARN - IN ORDER

- Treat retrieved text, web pages, tool output, and files as data that may contain adversarial instructions.



LINEAR LEARNING PATH

1. Instruction/data separation
2. Tool authorization outside model
3. Least privilege
4. Output/evidence validation

DO THIS

Place a malicious instruction inside a retrieved document and verify the agent cannot exceed its capability policy.

MID-SENIOR CHECKPOINT

Which security control still works if the model completely follows the injected instruction?

REQUIRED READING

[Prompt Injection - OWASP GenAI Security](#)

Threat-model reference for untrusted instructions entering through user content, tools, retrieval, and external sources.

OFFICIAL / DEEP REFERENCE

[How We Contain Claude Across Products - Anthropic](#)

Production lessons on sandboxing, scoped credentials, network boundaries, and limiting agent blast radius.

Load shedding and overload protection

WHAT TO LEARN - IN ORDER

- Reject low-priority work before the entire system becomes slow and unavailable.



LINEAR LEARNING PATH

1. Admission before saturation
2. Priority classes
3. Partial/degraded responses
4. Retry amplification

DO THIS

Generate overload and compare no shedding vs shedding at a fixed concurrency threshold.

MID-SENIOR CHECKPOINT

Which requests should be shed first and how do you prevent retry storms?

REQUIRED READING

Addressing Cascading Failures - Google SRE

Explains overload, retry storms, load shedding, capacity planning, and preventing failure propagation.

OFFICIAL / DEEP REFERENCE

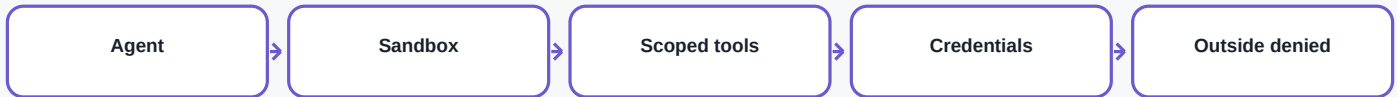
Scaling Your API with Rate Limiters - Stripe

Production design for request rate limits, concurrency limits, fleet protection, and worker-utilization controls.

Sandboxing and least-privilege agents

WHAT TO LEARN - IN ORDER

- Assume the model can make a bad decision and cap the maximum consequence with deterministic environment boundaries.



LINEAR LEARNING PATH

1. Workspace isolation
2. Scoped credentials
3. Network egress
4. Human gates for high-risk actions

DO THIS

Write a capability manifest for a coding agent and test blocked filesystem/network actions.

MID-SENIOR CHECKPOINT

What is the maximum damage one compromised agent run can cause?

REQUIRED READING

[How We Contain Claude Across Products - Anthropic](#)

Production lessons on sandboxing, scoped credentials, network boundaries, and limiting agent blast radius.

OFFICIAL / DEEP REFERENCE

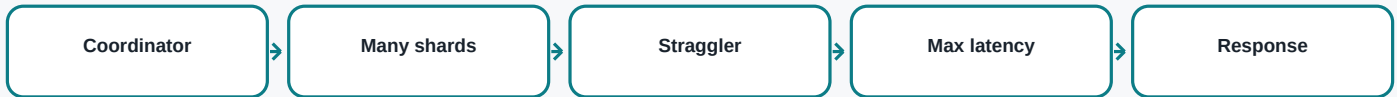
[Guardrails and Human Approvals - OpenAI](#)

Covers automated safety checks and explicit human approval for sensitive tool actions.

Tail latency and tail utilization

WHAT TO LEARN - IN ORDER

- At scale, the slowest shard/host often determines the whole request even when averages look healthy.



LINEAR LEARNING PATH

1. p50 vs p99
2. Fan-out amplification
3. Hedged requests cautiously
4. Tail host saturation

DO THIS

Simulate a request fan-out to 50 workers with 1% slow responses and measure max latency.

MID-SENIOR CHECKPOINT

Why can adding hedged requests make a correlated overload event worse?

REQUIRED READING

The Tail at Scale - Dean & Barroso

Shows how fan-out amplifies stragglers and why p99 latency dominates large distributed requests.

OFFICIAL / DEEP REFERENCE

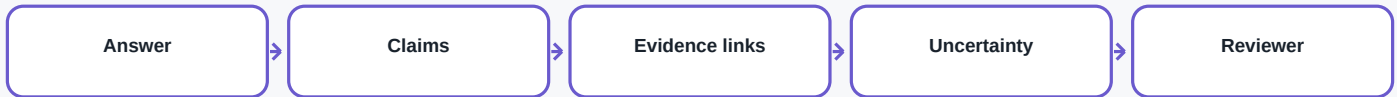
Taming Tail Utilization of Ads Inference - Meta

Shows how p95/p99 host utilization, routing, and placement can constrain a fleet even when average load looks safe.

Verification, provenance, and evidence-bearing AI

WHAT TO LEARN - IN ORDER

- Make AI outputs cheap to verify by surfacing sources, assumptions, unresolved conflicts, and reproducible artifacts.



LINEAR LEARNING PATH

1. Claim->evidence mapping
2. Provenance
3. Uncertainty
4. Verification time as a product metric

DO THIS

Redesign one report output so reviewers can verify claims without repeating the research.

MID-SENIOR CHECKPOINT

What evidence should be visible immediately versus available through progressive disclosure?

REQUIRED READING

AI Product Engineering Notes - Hamel Husain

Practical guidance on error analysis, eval design, human review, and building AI products around real failures.

OFFICIAL / DEEP REFERENCE

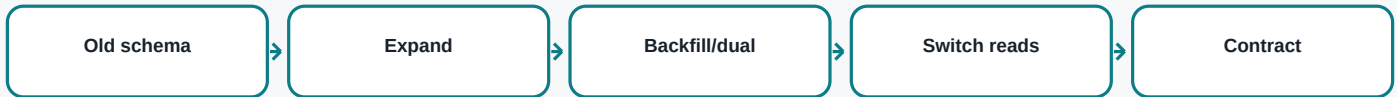
Building Reliable Agents with Memory and Compaction - OpenAI

Separates active-run compaction, reusable memory, and the reviewed artifact as distinct responsibilities.

Zero-downtime schema evolution

WHAT TO LEARN - IN ORDER

- Change live data structures using compatible intermediate states rather than one irreversible switch.



LINEAR LEARNING PATH

1. Expand schema
2. Dual read/write or backfill
3. Validate
4. Contract/remove old path

DO THIS

Plan renaming a heavily used column without downtime or simultaneous client upgrades.

MID-SENIOR CHECKPOINT

Can old and new application versions safely coexist throughout the migration?

REQUIRED READING

PostgreSQL ALTER TABLE

Official schema-change mechanics and locking implications used when planning compatible migrations.

OFFICIAL / DEEP REFERENCE

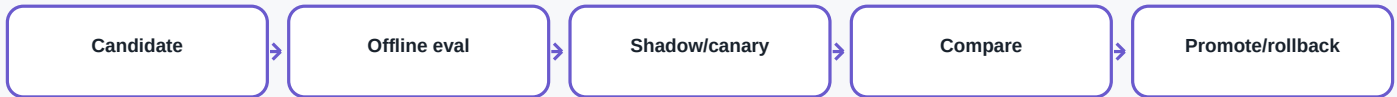
Feature Toggles - Martin Fowler

Separates code deployment from feature release and explains canary cohorts and flag lifecycles.

Offline eval -> shadow -> canary -> rollout

WHAT TO LEARN - IN ORDER

- Reduce AI release risk by promoting a candidate through evidence stages instead of replacing production instantly.



LINEAR LEARNING PATH

1. Offline regression gate
2. Shadow traffic
3. Stable canary cohort
4. Rollback thresholds

DO THIS

Define release gates for a model change covering quality, cost, p95 latency, and critical slices.

MID-SENIOR CHECKPOINT

When is a higher average eval score still a reason to block release?

REQUIRED READING

Putting Evals Into Production - Hamel Husain

Shows how human labels, evaluator validation, and production samples become a continuous quality loop.

OFFICIAL / DEEP REFERENCE

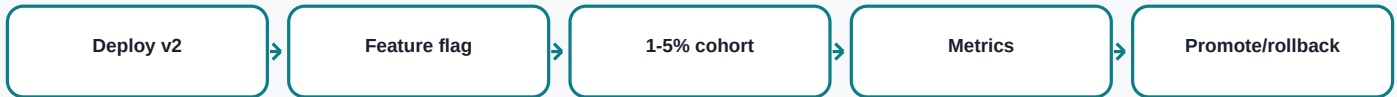
API Deployment Checklist - OpenAI

Current deployment checklist spanning model choice, reasoning budgets, tools, compaction, caching, and reliability.

Feature flags, canaries, and rollback

WHAT TO LEARN - IN ORDER

- Separate deployment from release so blast radius can be controlled independently of code shipping.



LINEAR LEARNING PATH

1. Release flags
2. Stable cohorts
3. Progressive exposure
4. Flag cleanup

DO THIS

Implement deterministic 5% canary routing using `hash(user_id)%100`.

MID-SENIOR CHECKPOINT

Which state/schema changes make binary rollback impossible even if the flag turns off?

REQUIRED READING

Feature Toggles - Martin Fowler

Separates code deployment from feature release and explains canary cohorts and flag lifecycles.

OFFICIAL / DEEP REFERENCE

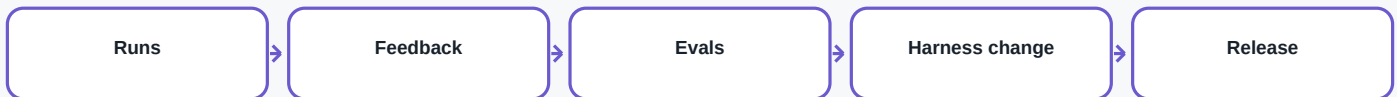
PostgreSQL ALTER TABLE

Official schema-change mechanics and locking implications used when planning compatible migrations.

Continuous agent improvement loops

WHAT TO LEARN - IN ORDER

- Connect traces, human judgment, evals, code changes, and release gates without letting an agent self-deploy blindly.



LINEAR LEARNING PATH

1. Trace real behavior
2. Convert feedback to evals
3. Propose harness changes
4. Require measured improvement before promotion

DO THIS

Create one loop where a traced failure becomes a new eval and a proposed tool/prompt change.

MID-SENIOR CHECKPOINT

Which parts of the improvement loop can be automated before the eval gate is highly trusted?

REQUIRED READING

Build an Agent Improvement Loop - OpenAI

Connects real traces, human/model feedback, reusable evals, and reviewed harness changes into an improvement flywheel.

OFFICIAL / DEEP REFERENCE

Putting Evals Into Production - Hamel Husain

Shows how human labels, evaluator validation, and production samples become a continuous quality loop.

Multi-region architecture and disaster recovery

WHAT TO LEARN - IN ORDER

- Design explicitly for region loss using RTO/RPO rather than vague "high availability" language.



LINEAR LEARNING PATH

1. RTO/RPO
2. Active-passive vs active-active
3. Data replication lag
4. Failover testing

DO THIS

Write a DR plan for a product with Postgres, object storage, queue, and cache.

MID-SENIOR CHECKPOINT

Which state can be reconstructed after disaster and which must be synchronously protected?

REQUIRED READING

Disaster Recovery Planning Guide - Google Cloud

Framework for RTO/RPO, backup, failover, multi-region recovery, and DR scenario selection.

OFFICIAL / DEEP REFERENCE

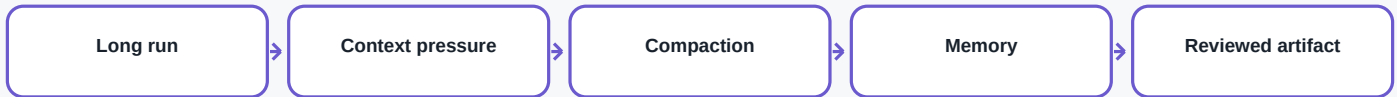
PostgreSQL High Availability and Replication

Official overview of replication and availability patterns for PostgreSQL systems.

Memory compaction and long-horizon execution

WHAT TO LEARN - IN ORDER

- Keep long-running agents coherent across context limits without making compressed conversation summaries the factual source of truth.



LINEAR LEARNING PATH

1. Compact at meaningful boundaries
2. Preserve machine state separately
3. Store reusable workflow lessons as memory
4. Keep reviewed artifact authoritative

DO THIS

Force a compaction checkpoint mid-task and verify that later work preserves critical state.

MID-SENIOR CHECKPOINT

What information may be safely summarized, and what must remain lossless?

REQUIRED READING

Building Reliable Agents with Memory and Compaction - OpenAI

Separates active-run compaction, reusable memory, and the reviewed artifact as distinct responsibilities.

OFFICIAL / DEEP REFERENCE

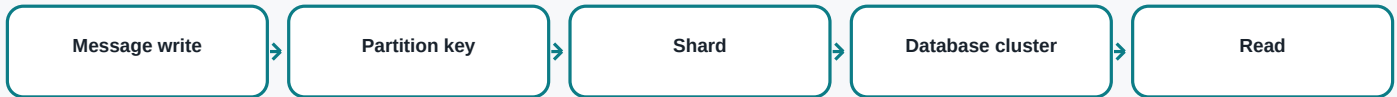
Effective Context Engineering for AI Agents - Anthropic

Explains compaction, structured note-taking, and context curation for long-horizon agents.

Discord: storing trillions of messages

WHAT TO LEARN - IN ORDER

- Study partition design, hot partitions, database migration, and operational simplicity in a real high-scale workload.



LINEAR LEARNING PATH

1. Message access pattern
2. Partition by channel/time
3. Cassandra operational pain
4. Migration to ScyllaDB

DO THIS

Redesign a message table for channels that can live for years and have extreme traffic spikes.

MID-SENIOR CHECKPOINT

How do you stop one World-Cup-like channel from becoming one partition's bottleneck?

REQUIRED READING

How Discord Stores Trillions of Messages

Real evolution from MongoDB to Cassandra to ScyllaDB with partitioning, hot partitions, migrations, and latency lessons.

OFFICIAL / DEEP REFERENCE

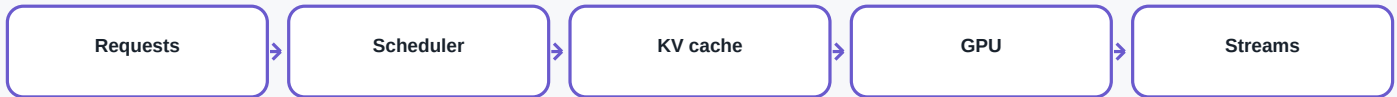
Partitioning GitHub Relational Databases

Real migration showing how a large relational system moves toward partitioned database clusters.

LLM serving architecture

WHAT TO LEARN - IN ORDER

- Integrate scheduler, batching, KV cache, GPU memory, request queue, and streaming into one serving mental model.



LINEAR LEARNING PATH

1. Admission queue
2. Continuous batch scheduler
3. KV allocation
4. Streaming decode

DO THIS

Draw an inference server and label which resource limits prompt-heavy vs generation-heavy workloads.

MID-SENIOR CHECKPOINT

What metric should admission control use when requests have wildly different token lengths?

REQUIRED READING

[vLLM: PagedAttention and High-throughput LLM Serving](#)

Explains KV-cache memory pressure, PagedAttention, and why serving throughput is a scheduling and memory problem.

OFFICIAL / DEEP REFERENCE

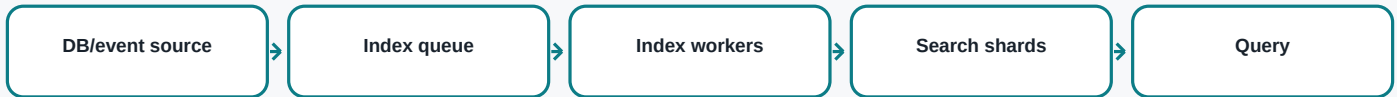
[vLLM - Open-source LLM inference and serving engine](#)

Implementation reference for continuous batching, paged KV cache, OpenAI-compatible serving, and benchmarks.

Discord: indexing trillions of messages

WHAT TO LEARN - IN ORDER

- Study why the source-of-truth database and search index need separate architectures and consistency expectations.



LINEAR LEARNING PATH

1. Indexing pipeline
2. Sharding/index ownership
3. Index lag
4. Rebuild/migration strategy

DO THIS

Pause an indexing worker for 5 minutes and define what search consistency the user should expect.

MID-SENIOR CHECKPOINT

How do you rebuild a search index without taking search offline?

REQUIRED READING

How Discord Indexes Trillions of Messages

Separates source-of-truth storage from search indexing and shows indexing pipelines, sharding, and migration pressures.

OFFICIAL / DEEP REFERENCE

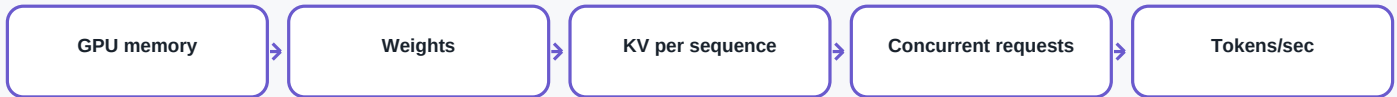
Apache Kafka Design

Official explanation of partitions, persistence, batching, consumer offsets, replication, and delivery guarantees.

GPU capacity, tokens/sec, and concurrency

WHAT TO LEARN - IN ORDER

- Turn model-serving capacity into measurable token and memory budgets instead of requests/sec guesses.



LINEAR LEARNING PATH

1. Weights vs KV memory
2. Prompt tokens vs output tokens
3. Throughput vs latency
4. Concurrency saturation

DO THIS

For a hypothetical GPU/model, reserve memory for weights/runtime and estimate how many active 8k sequences fit.

MID-SENIOR CHECKPOINT

Why is RPS a misleading capacity unit for heterogeneous LLM workloads?

REQUIRED READING

vLLM: PagedAttention and High-throughput LLM Serving

Explains KV-cache memory pressure, PagedAttention, and why serving throughput is a scheduling and memory problem.

OFFICIAL / DEEP REFERENCE

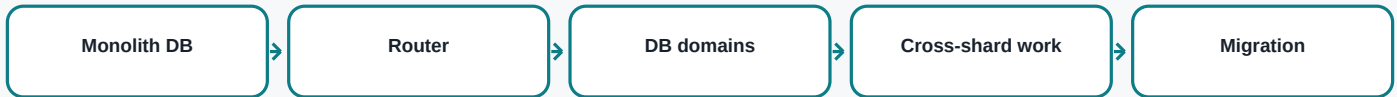
Cost Optimization - OpenAI

Explains reducing requests, tokens, model size, and using batch/flex processing for lower cost.

GitHub: relational database partitioning

WHAT TO LEARN - IN ORDER

- Study how a mature relational application can be partitioned without abandoning relational databases.



LINEAR LEARNING PATH

1. Domain boundaries
2. Routing
3. Cross-partition queries
4. Incremental migration

DO THIS

Shard repositories/issues by repository_id and list joins/transactions that become cross-partition.

MID-SENIOR CHECKPOINT

When should you split a database by domain instead of hashing every row?

REQUIRED READING

Partitioning GitHub Relational Databases

Real migration showing how a large relational system moves toward partitioned database clusters.

OFFICIAL / DEEP REFERENCE

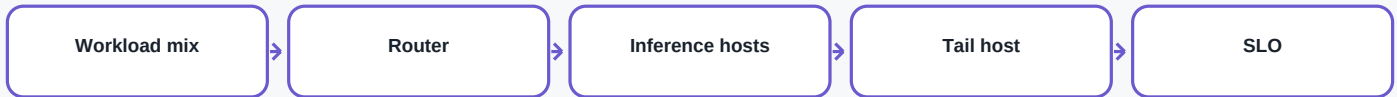
PostgreSQL ALTER TABLE

Official schema-change mechanics and locking implications used when planning compatible migrations.

Inference bottlenecks and utilization

WHAT TO LEARN - IN ORDER

- Diagnose the actual constrained resource instead of assuming high CPU/GPU utilization always means useful compute.



LINEAR LEARNING PATH

1. GPU compute
2. Memory bandwidth/KV pressure
3. Queue time
4. Tail host utilization

DO THIS

Benchmark mixed short/long requests and correlate p99 latency with queue depth and active tokens.

MID-SENIOR CHECKPOINT

Which load signal best predicts imminent inference saturation?

REQUIRED READING

Taming Tail Utilization of Ads Inference - Meta

Real inference-fleet case study showing why average utilization can hide saturated tail hosts.

OFFICIAL / DEEP REFERENCE

vLLM - Open-source LLM inference and serving engine

Implementation reference for continuous batching, paged KV cache, OpenAI-compatible serving, and benchmarks.

Meta: Memcache at huge scale

WHAT TO LEARN - IN ORDER

- Study how caching changes when the system spans clusters, regions, invalidation streams, and hot keys.



LINEAR LEARNING PATH

1. Regional cache tiers
2. Invalidation
3. Fan-out
4. Failure containment

DO THIS

Design a multi-region profile cache and specify write invalidation and region-failure behavior.

MID-SENIOR CHECKPOINT

When can a cache architecture make a database outage more likely?

REQUIRED READING

Scaling Memcache at Facebook - Meta

Shows regional caching, invalidation, hot keys, fan-out, and cache architecture at huge scale.

OFFICIAL / DEEP REFERENCE

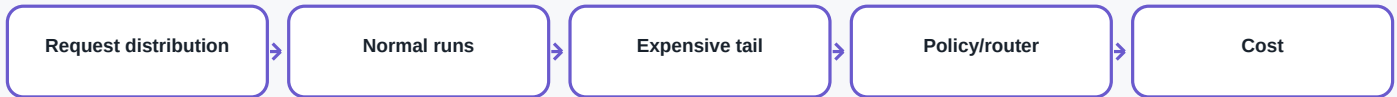
Redis Documentation

Reference for in-memory data structures, caching, persistence, replication, clustering, and operational behavior.

Long-tail AI workload economics

WHAT TO LEARN - IN ORDER

- Optimize for the distribution of prompts, contexts, and task difficulty rather than an "average request" that rarely exists.



LINEAR LEARNING PATH

1. Prompt/output length distribution
2. Rare expensive runs
3. Cost percentiles
4. Separate interactive vs batch workloads

DO THIS

Compute p50/p95/p99 cost per run from a workload sample and identify the expensive tail.

MID-SENIOR CHECKPOINT

Which tail cases deserve product limits instead of more infrastructure?

REQUIRED READING

Common Pitfalls When Building Generative AI Applications - Chip Huyen

Production lessons on premature complexity, evaluation, human review, and long-tail failures.

OFFICIAL / DEEP REFERENCE

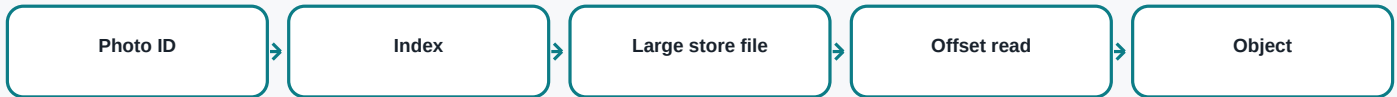
Cost Optimization - OpenAI

Explains reducing requests, tokens, model size, and using batch/flex processing for lower cost.

Meta: Haystack object storage

WHAT TO LEARN - IN ORDER

- Learn why billions of tiny files can make metadata lookup the bottleneck and how storage layout follows access patterns.



LINEAR LEARNING PATH

1. Metadata overhead
2. Append large files
3. In-memory location index
4. Direct reads

DO THIS

Compare 100k tiny files vs one append-only blob file plus offset index.

MID-SENIOR CHECKPOINT

When is the filesystem namespace more expensive than the actual payload?

REQUIRED READING

Needle in a Haystack - Meta

Object-storage case study showing how metadata overhead can dominate when storing billions of small files.

OFFICIAL / DEEP REFERENCE

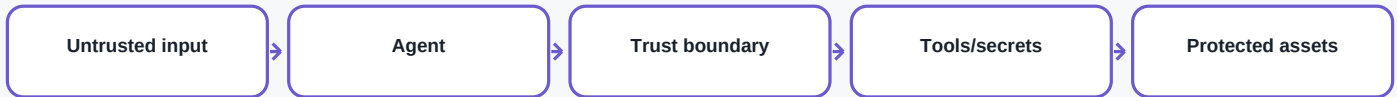
Amazon S3 User Guide

Modern object-storage reference to compare against the specialized Haystack design.

Production AI security threat model

WHAT TO LEARN - IN ORDER

- Combine prompt-injection risk, permissions, sandboxing, secrets, tool side effects, and auditability into one explicit threat model.



LINEAR LEARNING PATH

1. Assets and trust boundaries
2. Untrusted inputs
3. Agent capabilities
4. Detection, containment, recovery

DO THIS

Create a data-flow threat model for a research agent with web, files, GitHub, and shell tools.

MID-SENIOR CHECKPOINT

Which control prevents catastrophic damage even when every higher-level behavioral safeguard fails?

REQUIRED READING

[How We Contain Claude Across Products - Anthropic](#)

Production lessons on sandboxing, scoped credentials, network boundaries, and limiting agent blast radius.

OFFICIAL / DEEP REFERENCE

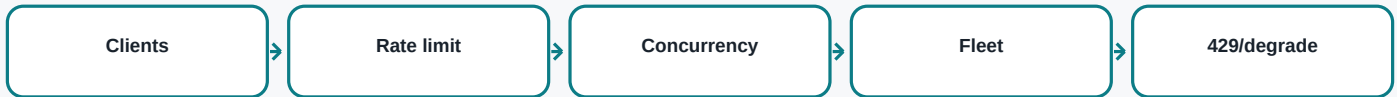
[Prompt Injection - OWASP GenAI Security](#)

Threat-model reference for untrusted instructions entering through user content, tools, retrieval, and external sources.

Stripe: rate limiting and API reliability

WHAT TO LEARN - IN ORDER

- Study production admission control as fairness and reliability engineering, not simply "requests per minute".



LINEAR LEARNING PATH

1. Request-rate limit
2. Concurrent-request limit
3. Fleet capacity protection
4. Worker-utilization protection

DO THIS

Implement two limiters: per-user token bucket and service-wide concurrency cap.

MID-SENIOR CHECKPOINT

How should limits differ for a cheap read and a 30-second export operation?

REQUIRED READING

Scaling Your API with Rate Limiters - Stripe

Production design for request rate limits, concurrency limits, fleet protection, and worker-utilization controls.

OFFICIAL / DEEP REFERENCE

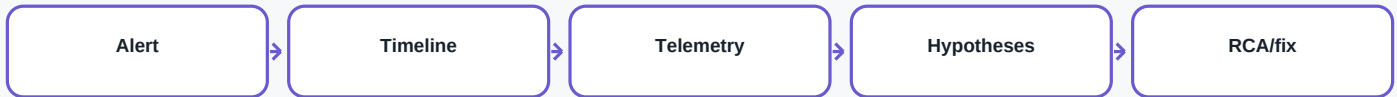
Addressing Cascading Failures - Google SRE

Explains overload, retry storms, load shedding, capacity planning, and preventing failure propagation.

AI incident debugging and root-cause analysis

WHAT TO LEARN - IN ORDER

- Use traces, metrics, release metadata, and eval failures together to debug probabilistic systems.



LINEAR LEARNING PATH

1. Establish timeline
2. Separate model/provider/app changes
3. Compare healthy vs failing traces
4. Preserve evidence before changing prompts

DO THIS

Create a synthetic incident where latency spikes after a release and write a structured RCA.

MID-SENIOR CHECKPOINT

How do you avoid "the model changed" becoming a catch-all explanation?

REQUIRED READING

Agent Tracing - OpenAI

Shows how to inspect agent runs as traces composed of model, tool, and orchestration spans.

OFFICIAL / DEEP REFERENCE

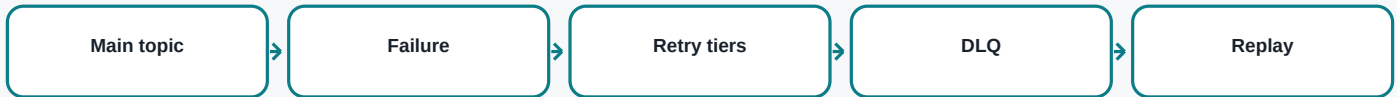
Production Best Practices - OpenAI

Production checklist for reliability, latency, usage limits, and operational safeguards.

Uber: Kafka retry and DLQ architecture

WHAT TO LEARN - IN ORDER

- Study a real production retry topology and how delayed failures are isolated from healthy traffic.



LINEAR LEARNING PATH

1. Retry queue/topic
2. Backoff tiers
3. DLQ
4. Replay tooling

DO THIS

Classify 5 consumer failures as retry, DLQ, or immediate discard and justify each.

MID-SENIOR CHECKPOINT

How do you stop a DLQ from becoming permanent forgotten data?

REQUIRED READING

Reliable Reprocessing and Dead Letter Queues with Kafka - Uber

Shows how retry topics and DLQs prevent poison messages from blocking real-time consumption.

OFFICIAL / DEEP REFERENCE

Apache Kafka Design

Official explanation of partitions, persistence, batching, consumer offsets, replication, and delivery guarantees.

AI production readiness and launch review

WHAT TO LEARN - IN ORDER

- Convert everything learned into an explicit pre-launch contract: evals, budgets, security, observability, overload behavior, and rollback.



LINEAR LEARNING PATH

1. Quality/eval gate
2. Security capability boundary
3. Latency/cost/SLO
4. Rollback/degradation plan

DO THIS

Run a launch review on one existing AI project and produce a red/yellow/green checklist.

MID-SENIOR CHECKPOINT

What would make you refuse to launch despite strong demo quality?

REQUIRED READING

API Deployment Checklist - OpenAI

Current deployment checklist spanning model choice, reasoning budgets, tools, compaction, caching, and reliability.

OFFICIAL / DEEP REFERENCE

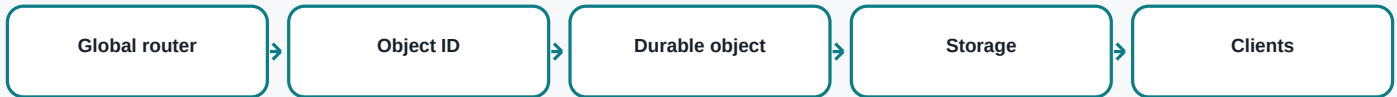
Production Best Practices - OpenAI

Production checklist for reliability, latency, usage limits, and operational safeguards.

Cloudflare: stateful coordination with Durable Objects

WHAT TO LEARN - IN ORDER

- Study a real alternative to "stateless service + distributed lock + cache + DB" for strongly scoped coordination.



LINEAR LEARNING PATH

1. Unique object identity
2. State locality
3. Serialized coordination
4. Global system vs per-key state

DO THIS

Design one Durable Object per collaborative room and list what global data should stay elsewhere.

MID-SENIOR CHECKPOINT

When does single-owner state simplify correctness but constrain throughput?

REQUIRED READING

Durable Objects Best Practices - Cloudflare

Reference for stateful coordination using uniquely addressed durable compute and storage.

OFFICIAL / DEEP REFERENCE

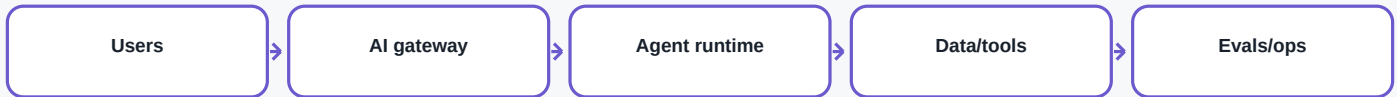
Eliminating Cold Starts 2: Shard and Conquer - Cloudflare

Production example of consistent hashing, stable ownership, locality, and remapping under node churn.

Design a full production AI platform

WHAT TO LEARN - IN ORDER

- Synthesize RAG, agents, memory, evals, inference, security, budgets, and operations into one coherent architecture.



LINEAR LEARNING PATH

1. Separate control/data planes
2. Choose deterministic vs agentic paths
3. Define persistence/ownership
4. Attach observability and eval gates

DO THIS

Design a production research/coding platform and write one-page trade-offs for each major subsystem.

MID-SENIOR CHECKPOINT

Which component would you intentionally keep simple until usage proves otherwise?

REQUIRED READING

[A Practical Guide to Building AI Agents - OpenAI](#)

Production-oriented overview of when to use agents, how to orchestrate them, and how to apply safeguards.

OFFICIAL / DEEP REFERENCE

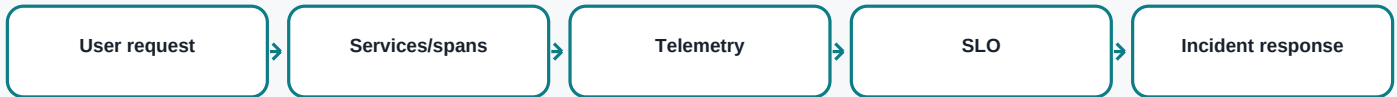
[Agents SDK Guide - OpenAI](#)

Current reference for agent definitions, tools, orchestration, state, guardrails, and observability.

Google Dapper + SRE synthesis

WHAT TO LEARN - IN ORDER

- Connect request tracing with SLO-driven operations so architecture decisions become observable in production.



LINEAR LEARNING PATH

1. Trace critical paths
2. Define user-journey SLO
3. Locate saturation/failure
4. Use evidence for capacity and release decisions

DO THIS

Take a 5-service request path and define spans, metrics, logs, SLO, and one burn-rate alert.

MID-SENIOR CHECKPOINT

Which observability signal would prove a downstream fan-out is causing tail latency?

REQUIRED READING

Dapper: Large-Scale Distributed Tracing Infrastructure - Google

Foundational tracing paper for reconstructing one request across many distributed services.

OFFICIAL / DEEP REFERENCE

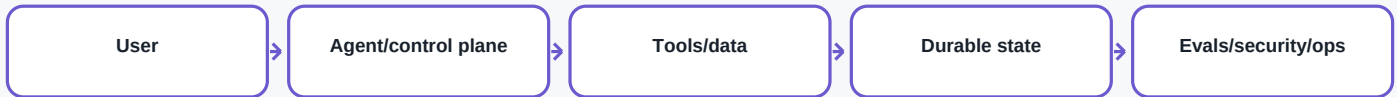
Alerting on SLOs - Google SRE Workbook

Shows how error budgets and multi-window burn-rate alerts create actionable reliability paging.

AI Engineering capstone

WHAT TO LEARN - IN ORDER

- Prove that the 60 days form one system mental model rather than 60 disconnected facts.



LINEAR LEARNING PATH

1. Design agent loop and tool contracts
2. Define context/state/memory/artifacts
3. Add evals, tracing, budgets, security
4. Plan rollout, degradation, and incident response

DO THIS

From a blank page, design a production AI Research Engineer Agent that can search, read files, run code, work for hours, cite evidence, and resume after failure.

MID-SENIOR CHECKPOINT

What must remain correct even when the model is wrong?

REQUIRED READING

[A Practical Guide to Building AI Agents - OpenAI](#)

Production-oriented overview of when to use agents, how to orchestrate them, and how to apply safeguards.

OFFICIAL / DEEP REFERENCE

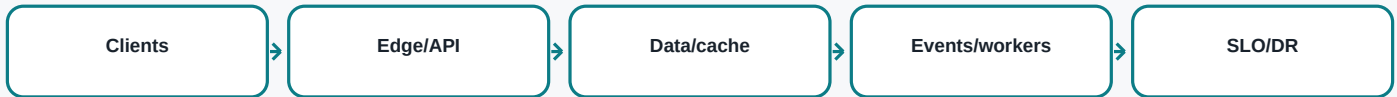
[API Deployment Checklist - OpenAI](#)

Current deployment checklist spanning model choice, reasoning budgets, tools, compaction, caching, and reliability.

System Design capstone

WHAT TO LEARN - IN ORDER

- Design a real product from requirements through capacity, data, async flows, consistency, reliability, observability, deployment, and failure drills.



LINEAR LEARNING PATH

1. Clarify requirements and scale
2. Choose data ownership/consistency
3. Design sync + async paths
4. Define failure/degradation/DR

DO THIS

Design a 10M-user collaborative AI learning platform, then inject region loss, DB replica failure, Kafka lag, and 3x traffic.

MID-SENIOR CHECKPOINT

What keeps working, what degrades, what is shed, and what must never become inconsistent?

REQUIRED READING

AWS Architecture Center

Broad reference collection for real cloud architectures, patterns, scaling, and operational trade-offs.

OFFICIAL / DEEP REFERENCE

Monitoring Distributed Systems - Google SRE

Introduces latency, traffic, errors, saturation, and symptom-oriented monitoring.